

--ctrl + r to toggle the results window

SELECT QUERY

/*

ORDER OF THE SELECT QUERY

SELECT

DISTINCT

TOP

FROM

WHERE

GROUP BY

HAVING

ORDER BY

*/

--you can use the below command to choose your desired database

use MyDatabase

--Selecting all the data from the table

*select **

from customers

*select **

from orders

--Selecting only required columns from the table

select

country,

score

from customers

WHERE QUERY

--only getting the data having score > 500

*select **

from customers

where score > 500

--Getting the data where score > 0

*select **

from customers

where score != 0

--Getting the data where country is germany

*select **

from customers

where country = 'Germany'

ORDER BY QUERY

--ordering the results in increasing order

*select **

from customers

order by score asc

--ordering the results in decreasing order

*select **

from customers

order by score desc

--order by (nested)

*select **

from customers

order by country asc,

score desc

GROUP BY

--grouping on same country

select

country,

sum(score) as

totalscore

from customers

group by country

order by country desc

/*GROUP BY RULE

all columns in the SELECT must be either aggregated or included in the GROUP BY*/

--finding the total score and total number of customers for each country

select

country,

COUNT(id) as

total_customers,

SUM(score) as

Total_Score

from customers

group by country

order by Total_Score desc

HAVING CLAUSE

--filters data after aggregation

--filters end results

--can only be used after using the GROUP BY

--used to filter the end results

--finding the total customers and total score for each country (having the score > 500)

select

country,

COUNT(id) as

Total_customers,

SUM(score) as

Total_Score

from customers

group by country

having SUM(score) > 500

order by Total_Score desc

EXPLANATION : The above query selects the country and count of customers id to find the total number of customers having alias Total_customers. The sum(score) finds the sum of all the score having alias Total_Score. The group by groups the results based on country (all the customers in a country will be sum and shown in column(Total_customers) and the results will be ordered by total_score decreasing order

--finding the total customers and total score for each country (having the score < 860)

select

country,

COUNT(id) as

Total_customers,

SUM(score) as

Total_Score

from customers

group by country

having SUM(score)<860

order by Total_Score desc

--finding the avg score for each country considering only the customers with a score not equal to 0

--and returning only the country with an avg score > than 430

select

country,

AVG(score) as

AVG_Score

from customers

where score!=0

group by country

having AVG(score)>430

order by AVG_Score desc

EXPLANATION : the above query selects the country and aggregates the score by finding average. The where score!=0 filters out costumers having

score as 0 (means they will not be considered in the answer).

The group by groups the results by country (means the output will have average per country). Having avg(score)>430 filters the end results means only the country with an avg score above than 430 will be visible in the results. The order by orders on the basis of decreasing order avg_score.

DISTINCT

--removes repeated values (duplicates)

--each value appears only once

--dont use DISTINCT unless necessary (it can slow down our query)

--returning unique list of all the countries

select

DISTINCT country

from customers

TOP(LIMIT)

--restricts the number of results returned in the results

--retrieving top 3 customers with the highest scores

*select top 3 **

from customers

order by score desc

--retrieve the top 2 customers with the lowest scores

```
select top 2 *
from customers
order by score asc
```

--retrieve the two most recent orders

```
select top 3 *
from orders
order by order_date asc
```

DDL COMMANDS

(DATA DEFINITION LANGUAGE)

--DDL commands doesn't return data it changes the structure of data

```
use MyDatabase
```

--CREATE

--creating a new table called persons

--with columns :id, person_name, birth_date, and phone

```
create table persons (
    ID int not null,
    Person_Name
varchar(30) not null,
    Birth_Date Date,
```

```
    phone varchar(15) not
null,
    CONSTRAINT pk_key
primary key (ID)
)
```

```
select * from persons
```

--ALTER

--is used when we want to edit our definition of the table

--to add multiple cols only 1 add can be used

--use of multiple add will lead to error

--the alter adds the column to the end of the table if we wish to place the column in between we will need to recreate the table

```
ALTER table persons
```

```
add pincode char(6) not null,
email varchar(50) not null;
```

```
select *
from persons
```

```
ALTER table persons
drop column pincode,
email;
```

--DROP

--is used to delete the table or database or columns too

```
drop table persons
```

DML COMMANDS

(DATA MANIPULATION LANGUAGE)

```
use MyDatabase
```

--INSERT

--inserting data into a table

--if in the insert statement no cols are specified the sql expects values for all the cols

--you can skip the column names if you are inserting into every column

```
select * from customers
```

```
insert into customers (id,
first_name, country, score)
values
```

```
(6, 'Shubham Sharma', 'India',
580),
```

```
(7, 'Nitin Sharma', 'India',
680);
```

```
insert into customers (id,
first_name, country)
values
```

```
(8, 'Mohit Sharma', 'India')
```

--INSERT USING SELECT

/*

In the SELECT clause, each item can be:

1. A column name
2. table.* to select all columns
3. Any expression (constants like NULL or 'UNKNOWN', functions, arithmetic, etc.)

*/

--Inserting data from a source table to target table

--selecting the data needed according to the structure of the persons table

select

id,

first_name,

null,

'UNKNOWN'

from customers

--just adding an insert into to insert the selected data form the customers table to persons table

insert into persons (id, person_name, birth_date, phone)

select

id,

first_name,

null,

'UNKNOWN'

from customers

EXPLANATION: The above query inserts the selected data from the customers table into the target table persons. The id from the customers is selected, the first_name from the customers is also selected, the next columns named null will contain a column having null values in the birth_date column of persons, the next column named unknown will fill the column with unknown in the place of phone

--checking if the data is inserted successfully

select *

from persons

UPDATE STATEMENT

use MyDatabase

--update

--is used in order to change the pre existing content in the table

--TIP (always use where to avoid updating all the rows unintentionally)

/*

Update table_name

set col1 = value1,

col2 = value2

where <condition>

*/

--changing the score of customer having id 6 to 0

--checking the data before updation

select *

from customers

--updating

update customers

set score = 0

where id = 6

--checking the updation result

select

id,

score

from customers

where id = 6

--changing the score of customer having id 8 to 0 and updating the country to UK

--checking the data before updation

select *

from customers

--updating

update customers

set score = 0,

country = 'UK'

where id = 8

--checking the updation result

```

select *
from customers
where id = 8

--updating the columns having
null score to 0

--checking the data before
upadation
select *
from customers
--updating
update customers
set score = 0
where score IS NULL
--checking the upadation result
select *
from customers
use MyDatabase

```

DELETE

```

--is used to delete existing
rows in your table
/*
Delete from table_name
where <condition>
*/

--deleting all the customers
having id greater than 5
delete from
customers
where id > 5
--checking the deletion result

```

```

select *
from customers
where id > 5 --this will give
null if all the id's greater than 5
are removed

```

TRUNCATE

```

--is used to delete all content
from a table in one go

--it just deletes all the table
content and not drops the table

```

```
truncate table persons
```

WHERE OPERATORS

```
use MyDatabase
```

--COMPARISON OPERATORS

```
--(=), (<>, !=), >, <, >=,
<=
```

```
--retrive all the customers
from germany
```

```
select *
from customers
where country = 'Germany'
```

```
--retrieve all the customers
who are not fromm germany
```

```
--(<> and != work the same in
microsoft sql server)
```

```
select *
from customers
where country!='Germany'
```

```
--retrive all the customers
where score is greater than 500
```

```
select *
from customers
where score>500
```

```
--retrive all the customers
where score is greater than or
equal to 500
```

```
select *
from customers
where score>=500
```

```
--retrive all the customers
where score is lesser than 500
```

```
select *
from customers
where score<500
```

```
--retrive all the customers
where score is lesser than or
equal to 500
```

```
select *
from customers
where score<=500
```

--LOGICAL OPERATOR

--AND OR NOT

use MyDatabase

--AND

--all the conditions must to be true to return true

--retrive all the customers
having country has USA and
score > than 500

*select **

from customers

where country='USA'

and

score>500

--OR

--at least one conditions must be true to return true

--retrieve all the customers
either having usa as country or
score > 500

*select **

from customers

where country='USA'

or

score>500

--NOT

--retrives all the data not fulfilling the conditions

--it reverses the whole answer

--only gives the answer not fulfilling the condition

--retrieve all the customers
having score not less than 500

*select **

from customers

where not score>500

--using with AND condition

--retrieving all the customers
data having score not less than
500 and country not having
USA

*select **

from customers

*where NOT (score>500) and
NOT (country='USA')*

--RANGE OPERATORS

use MyDatabase

--BETWEEN

--used to find the value between the two values

--the values specified are inclusive

--if the value are between the specified values then it will return true otherwise false

--retrieve all the customers
whose score falls in the range
bw 100 and 500

*select **

from customers

where score

between 100 and 500

--another way to to the same

*select **

from customers

where score>=100

and

score<=500

use MyDatabase

--MEMBERSHIP OPERATORS

--IN / NOT IN

--used to give list as or single element as input to check if it exists in the table

--use in instead of OR for multiple values in the same column to simplify SQL

--retrieve all the customers
from either germany or usa

*select **

from customers

*where country in ('Germany',
'USA')*

--another way

*select **

from customers

*where country='Germany' or
country='USA'*

--retrieve all the customers not from germany, India, africa

*select **

from customers

where country not in ('Germany', 'India', 'Africa')

--LIKE OPERATOR

--is used to search for a pattern inside the text

--% is used (0, 1, MANY)

--The _ is used if exact 1 value is needed to find

--(% before the text means)=>anything in the start and specified characters at the last

--For example a name - MARTIN (to find the last 'in' we have to write %in) which means starting can be any but the ending must be in

--To find the 'MA' we will write MA% which means ending can be any but the starting should be from MA

--to find the 'R' in the MARTIN we will write %R% which means R anywhere in the text

use MyDatabase

--finding all the customers name starting with M

*select **

from customers

where first_name like 'M%'

--finding all the customers name ending with n

*select **

from customers

where first_name like '%n'

--finding all the customers name containing r

*select **

from customers

where first_name like '%r%'

--find all customers whose first name has r in the 3rd pos

*select **

from customers

where first_name like '__r%'

--NOT LIKE OPERATOR

--finding the customers names not starting with M

*select **

from customers

where first_name not like 'M%'

--finding all the customers name not ending with n

*select **

from customers

where first_name not like '%n'

JOINS

--used in combining the columns of the tables

--Basic types of joins - NO, INNER, FULL, RIGHT, LEFT

--Advanced joins - LEFT ANTI JOIN, RIGHT ANTI JOIN, FULL ANTI JOIN, CROSS

--In order to join the tables we need to define the key cols between the two tables

--the joins can be performed if their is a column which is in both the tables we want to join

WHEN TO USE WHICH JOIN: -

★DON'T USE RIGHT JOIN

1. ONLY MATCHING – INNER JOIN
2. ALL ROWS
 - (ONE SIDE) LEFT JOIN
 - (BOTH SIDE) FULL JOIN
3. ONLY UNMATCHING
 - (ONE SIDE) LEFT ANTI JOIN
 - (BOTH SIDE) FULL ANTI JOIN

use MyDatabase;

★--TIP -> always stick with the left join (as both right and left join work similarly)

★--just keep in mind the table name placement before left join or after the left join

--NO JOIN

--returns data flow tables without combining them

--two results no need to combine

--retrieve all data from customers and orders as separate results

*select **

from customers;

*select **

from orders;

--INNER JOIN

--returns only the matching rows from both the tables (only overlapping data)

--all customers data along with their orders (but only for customers who have placed an order)

--uses - recombine data(combine multiple tables), data enrichment(extra info about data), check existence(filtering)

select

a.id,

a.first_name,

a.country,

a.score,

b.sales

from customers as a

inner join orders as b

on a.id = b.customer_id

where b.sales!=0;

--LEFT JOIN

--returns all the rows from the left table and only matching data from the right table

--left table is the primary source of data and the right table is secondary source of data (only for additional data)

--get all customers along with their orders including those who didn't ordered

--here in the left table is and right table is b

select

a.id,

a.first_name,

b.sales

from customers as a

left join orders as b

on a.id = b.customer_id

--getting the customers data who didnt ordered

select

a.id,

a.first_name,

b.sales

from customers as a

left join orders as b

on a.id = b.customer_id

where b.customer_id is null

--RIGHT JOIN

--returns all the rows from the RIGHT table and only matching data from the left table

--Right table is the primary source of data and Left table is secondary source of data (only for additional data)

--get all customers along with their orders,including orders without matching customers

select

a.customer_id,

a.sales,

b.first_name,

b.country,

b.score

from orders as a

right join customers as b

on a.customer_id = b.id

--same quesiton with left join

select

a.customer_id,

```

a.sales,
b.first_name,
b.country,
b.score

from customers as b
left join orders as a
on b.id = a.customer_id

```

--FULL JOIN

--returns all the rows from both the tables

```

select *

from customers as a
full join orders as b
on a.id = b.customer_id

```

ADVANCED JOINS

--LEFT ANTI JOIN (left exclusive join)

--uses -> just for checking existence (filtering)

--return the rows from the left which has no match in the right

--the matching data will is not shown in the results

--only that data will be shown which is exclusively in the left table and is not in the right table

--in this the the left table (a) is the primary source of data and the right table is just for filter

use MyDatabase

--retrieve all the customers who haven't ordered

```

select *

from customers as a
left join orders as b
on a.id = b.customer_id
where b.customer_id is null

```

--RIGHT ANTI JOIN (right exclusive join)

--returns the rows from the right which has no match in the left table

--retrieve all the orders not having customer details in the customers table

```

select *

from customers as a
right join orders as b
on a.id = b.customer_id
where a.id is null

```

--we should use the left anti join to do the work of right anti join also

--retrieve all the orders not having customer details in the customers table with leftantijoin

```

select *

from orders as a
left join customers as b
on b.id = a.customer_id
where b.id is null

```

--FULL ANTI JOIN (FULL EXCLUSIVE JOIN)

--uses -> checking existence (filtering)

--only returns rows that don't match in either tables

--overlapping data between both the tables will not be shown

--find customers without orders and orders without customers

--here we used or operator because if we use and it will need both the conditions to be true which is not possible

```

select *

from customers as a
full join orders as b
on a.id = b.customer_id
where b.customer_id is null
or
a.id is null

```

--challenge

--get all customers along with their orders but only for

customers who have placed an order without using an inner join

--by left join

*select **

from orders as a

left join customers as b

on b.id = a.customer_id

where b.id is not null

--by inner join

*select **

from customers as a

inner join orders as b

on a.id = b.customer_id

--by full join

--getting all the orders

*select **

from customers as a

full join orders as b

on a.id = b.customer_id

where b.customer_id is not null

and

a.id is not null

--CROSS JOIN

--combines every row from the left and every row from the right

--very rarely used

--in every possible ways

--the output will be -> the rows in the table a x table b.

--generate all possible combinations of customers and orders

*select **

from customers

cross join orders

--JOINTING TWO OR MORE TABLES

--their can be a master table means the main tables

--the other tables will be for extra information

use SalesDB

--using the salesdb, retrieve a list of all orders,

--along with the related customer, product, and emp details

--here the master table is the orders table

select

O.OrderID,

O.Sales,

O.CustomerID,

c.FirstName,

c.LastName,

c.Country,

c.Score,

p.Product,

p.Category,

e.EmployeeID as SalesPersonID,

e.FirstName as SalesPersonName,

e.ManagerID

from Sales.Orders as o

left join Sales.Customers as c

on o.CustomerID = c.CustomerID

left join Sales.Products as p

on o.ProductID = p.ProductID

left join sales.Employees as E

on e.EmployeeID = o.SalesPersonID

--SET operators

--used in combining the rows of the table

--Basic types of set operators - UNION, UNION ALL, EXCEPT(MINUS), INTERSECT

--To combine rows with set operators tables in the query should has the exact same number of columns

--set operators dont need key cols like the join query to join the tables

--the set operators appends the data of the table b beneath the data of table a

/*

RULES :-

1. Set operators can be used almost in all the clauses

2. ORDER BY is only allowed once at the end of the query

3. The number of cols in each query must be the same

4. Datatypes of cols in each query must be compatible (or same)

5. The order of the cols in each query must be the same

6. The col names in the result are determined by the column names set in the first query

(means the first query will be controlling the names of the column in the results, and col names

given to the second query will be totally ignored)

*/

--append the employee names beneath the names of customer names

--rule no 1 and 2 (order by can be used only once at the last)

select

*FirstName,
LastName
from Sales.Customers*

UNION

select

*FirstName,
LastName
from Sales.Employees*

order by FirstName asc

--rule no 3 (no of cols in each query must be the same)

--the below query will not work as no of cols in each column are not the same

select

*c.CustomerID,
c.FirstName,
c.LastName
from Sales.Customers as c*

UNION

select

*e.FirstName,
e.LastName
from Sales.Employees as e*

--rule no 4 (Datatypes of cols in each query must be compatible (or same))

--in this the conversion from varchar will be failed

select

*c.CustomerID, --
different data type from the
first name*

*c.LastName
from Sales.Customers as c*

UNION

select

*e.FirstName,
e.LastName
from Sales.Employees as e*

--rule no 5 (The order of the cols in each query must be the same)

--the query below will not work as the order is not correct in both the queries

select

*c.FirstName,
c.CustomerID
from Sales.Customers as c*

UNION

select

*e.EmployeeID,
e.FirstName
from Sales.Employees as e*

/*RULE NO 6 The col names in the result are determined by the column names set in the first query

(means the first query will be controlling the names of the column in the results, and col names

given to the second query will be totally ignored)*/

select

c.FirstName as Name,

c.LastName as

Surname

from Sales.Customers as c

UNION

select

e.FirstName,

e.LastName

from Sales.Employees as e;

--RULE NO 7

/*Even if all rules are met and SQL shows no error, the result may be incorrect

(Incorrect column selection leads to inaccurate results)*/

--here we didn't got any error but the answer is incorrect

--because in the second query the last name is written on the place of firstname

--and it will change the results and put last name on the place of firstname

select

c.FirstName as Name,

c.LastName as

Surname

from Sales.Customers as c

UNION

select

e.LastName,

e.FirstName

from Sales.Employees as e

--UNION

--returns all distinct rows from both the queries

--remove duplicate results from the result

--means if (Firstname - Shubham, Lastname - Sharma) is present in the first query

--(Firstname - Shubham, Lastname - Sharma) is also present in the last query

--the one result will be removed from this and only one will be visible in the result

--COMBINE the data from employees and customers into one table

--checking first which information we can map together

*select **

from Sales.Employees

*select **

from Sales.Customers

--combining

select

c.CustomerID,

c.FirstName,

c.LastName

from Sales.Customers as c

UNION

select

e.EmployeeID,

e.FirstName,

e.LastName

from Sales.Employees as e

--UNION ALL

--returns all rows from both the queries, including duplicates

--its faster than union because it dont need to find for duplicates

--use union all to find duplicates and quality issues

--combine the customers table and employees table including duplicates

--this will include duplicates too

select

c.CustomerID,

c.FirstName,

c.LastName

from Sales.Customers as c

UNION ALL

select

e.EmployeeID,

e.FirstName,

e.LastName

from Sales.Employees as e

--EXCEPT (MINUS)

--returns all distinct rows from the first query that are not found in the second query

--the 2nd query just works as a filter (the results will not have 2nd query elements)

--its the only one where the order of the queries affects the final result

--ITS LIKE A LEFT ANTI JOIN

*/**

EXCEPT works only on the first query's results. The second query is used purely as a filter:

- Step 1: Take all rows from the first query.

- Step 2: Look at the second query.

- Step 3: Remove any rows from Step 1 that also appear in Step 2.

- Step 4: Done you never see rows from the second query itself.

**/*

--find the employees who are not customers at the same time

select

c.CustomerID,

c.FirstName,

c.LastName

from Sales.Customers as c

EXCEPT

select

e.EmployeeID,

e.FirstName,

e.LastName

from Sales.Employees as e

--INTERSECT

--returns only the rows which are common in both the queries

--order of the queries doesn't affect the results

--ITS LIKE INNER JOIN

--find the employees who are also customers

select

e.EmployeeID,

e.FirstName,

e.LastName

from Sales.Employees as e

INTERSECT

select

c.CustomerID,

c.FirstName,

c.LastName

from Sales.Customers as c

--SET operators uses

--USES OF SET OPERATORS

*/**

1. Combining similar info before analyzing the data

I. Normally what we do is writing separate sql queries

for making a report of
INDIVIDUALS

II. Whether they are
customers, employees,
suppliers, students (getting
diff-diff tables for all of them)

which makes report
untidy)

III. What we can do is
combine all of them using
UNION and making just 1
table that holds all the info

for individuals

2. Combining multiple tables
divided into multiple tables by
developers to optimize
performance

I. Like multiple tables
having data for differnt
different year orders (like
order 2021, order 2022 and so
on)

*/

--tips

--NEVER USE * TO
COMBINE TABLES

--you can use directly the
names of the cols list (by right
clicking on the table in obj
explorer and select top
1000rows)

--in that query will be having
the full list of col names just
simply copy and paste

--SHIFT + TAB to remove tab
character before the col names

--because if the

--Orders are stored in separate
tables (orders and
ordersArchive). Combine all
into one report without
duplicates

--checking tables before
combining

*select **

from Sales.Orders;

*select **

from Sales.OrdersArchive;

--combining

select

*'Orders' as SourceTable --this
will add a new col in the result
(showing the source of the
table)*

,[OrderID]

,[ProductID]

,[CustomerID]

,[SalesPersonID]

,[OrderDate]

,[ShipDate]

,[OrderStatus]

,[ShipAddress]

,[BillAddress]

,[Quantity]

,[Sales]

,[CreationTime]

from Sales.Orders

UNION

select

*'OrdersArchive' as
SourceTable*

,[OrderID]

,[ProductID]

,[CustomerID]

,[SalesPersonID]

,[OrderDate]

,[ShipDate]

,[OrderStatus]

,[ShipAddress]

,[BillAddress]

,[Quantity]

,[Sales]

,[CreationTime]

*from Sales.OrdersArchive as
oa*

order by OrderDate desc

--EXCEPT USE CASES

--1.DELTA DELETION

*/**

We use the EXCEPT in order
to indentify the differences or
changes(delta) between two
batches of data

*/**

--data engineers build data
pipelines in order to load daily
new data from source to
datawarehouse or datalake

*/**

like on the day 1 we get a data

1, JohnDoe, john@gmail.com,
2024-09-17

2, Jan Doe, jan@outlook.com,
2024-09-18

on the day 2 we get

1, JohnDoe, john@gmail.com,
2024-09-17

3, Alice, alice@outlook.com,
2024-09-19

now the day 1 data is already
uploaded to the data
warehouse

so that duplicates are not
uploaded to the data
warehouse

we can use EXCEPT

which will filter out the data
which is already existing in the
data warehouse

*/

--2.Data completeness check

/*

can be used to compare tables
to detect discrepancies
between databases

like we migrated a data from
database a to database b

now to check that all the data
is successfully transferred from
a to b

we will check with the
EXCEPT operator

IF ALL THE DATA IS
TRANSFERRED
SUCCESSFULLY THE
RESULT WILL BE NULL

same we can do vice versa

checking b to a if there are
records which are in b and not
in a

*/

--QUICK SUMMARY SET
OPERATORS

/*

TYPES: - UNION, UNION
ALL, EXCEPT, INTERSECT

RULES: - Same no of cols,
Datatypes, order of cols / 1st
query controls the col names

USES: - Combine
Info(UNION + UNION ALL),
Delta Deletion(EXCEPT),
Data Completeness(EXCEPT)

*/

--FUNCTIONS

/*

is a built in sql code:

1. which accepts an
input value

2. processes it

3. returns an output
value

*/

--2 types of built in functions

--SINGLE ROW
FUNCTIONS - used for
cleanup, transformation,
manipulation (like LOWER())

/*SRF FULL LIST - STRING,
NUMERIC, DATE&TIME,
NULL*/

--MULTI ROW FUNCTIONS
(like SUM())

/*MRF FULL LIST -
AGGREGATE, WINDOW*/

--NESTED FUNCTIONS?

--are functions used inside
another function

--write a function to get the
first two letters from the id 1
firstname in lowercase

select

left(lower(first_name),
2)

from

Mydatabase.dbo.customers

where id = 1;

--STRING FUNCTIONS

/*

MANIPULATION -
CONCAT, UPPER, LOWER,
TRIM, REPLACE

CALCULATION - LEN

STRING EXTRACTION -
LEFT, RIGHT, SUBSTRING

*/

--MANIPULATION**--concat**

--combines multiple strings into one

--concatenate first name and country into one column

select

c.FirstName,

c.Country,

CONCAT(c.FirstName, '-' ,c.Country) as name_country

from Sales.Customers as c

--UPPER / LOWER

--convert all characters of a str to upper or lowercase

--convert the customers firstname to lowercase and upper case

select

c.FirstName,

LOWER(c.FirstName)

as Firstname_lower,

UPPER(c.FirstName)

as Firstname_upper

from Sales.Customers as c

--TRIM

--removes the leading and trailing empty whitespaces

--find the customers whose firstname contains leading or trailing spaces

select

c.FirstName

from Sales.Customers as c

where c.FirstName !=

TRIM(c.FirstName)

--same question with different way

select

FirstName,

LEN(FirstName) as

Name_len,

LEN(TRIM(FirstName)) as Name_len_trimmed,

LEN(FirstName) - LEN(TRIM(FirstName)) as Spaces

from Sales.Customers

where LEN(FirstName) !=

LEN(TRIM(FirstName))

--REPLACE

--replaces specific character with a new character

--remove the - from the number

select

'123-456-7890' as phone,

REPLACE('123-456-7890', '-', '') as clean_phone

--replace the - from the number to '/'

select

'123-456-7890' as

phone,

REPLACE('123-456-7890', '-', '/') as clean_phone

--replace the file ext (.txt) to (.docx)

select

'report.txt' as

old_filename,

REPLACE('report.txt', '.txt', '.csv')

--CALCULATION - LEN

--LEN

--counts the number of characters / digits / dates anything and return it in the form of integer

--count the no of characters in the name

select

FirstName as Name,

LEN(FirstName) as

Name_Length

from Sales.Customers

--count the no of digits in date

--this will count the no of characters even the dashes

select

'2025-08-30' as Date,

*LEN('2025-08-30') as
Date_Length*

--STRING EXTRACTION - LEFT, RIGHT, SUBSTRING

--LEFT / RIGHT

--extracts specific no of
characters from the start or
from the end respectively

--extract the first 3 char of
each customer's firstname

select

FirstName as Name,

*left(FirstName, 3) as
short_name*

from Sales.Customers

--extract the last 3 char of each
customer's last name

select

LastName,

*RIGHT(LastName, 3)
as short_surname*

from Sales.Customers

--where *LEN(LastName) != 0*
can be added if we dont want
the data not having lastnames

--SUBSTRING

--extract 2 characters after the
2nd char of each customer's
firstname

select

FirstName,

*SUBSTRING(FirstNam
e,2, 2)*

from Sales.Customers

--extract all the characters after
the 2nd char of each
customer's firstname

--here we have use len because
after 2 characters the len will
be more than the len of the
remaining characters

--this will help us to get all the
characters after the 2nd char

--entering length more than the
characters left will return all
the characters and will not give
an error

select

FirstName,

*SUBSTRING(FirstNam
e,2, LEN(FirstName))*

from Sales.Customers

--extract all the characters after
the 2nd char of each
customer's firstname without
whitespaces

select

FirstName,

*SUBSTRING(TRIM(Fir
stName), 2, LEN(FirstName))
as sub_name*

from Sales.Customers

--NUMBER FUNCTIONS

--ROUND

--will round the value based on
its value after the specified
decimal place

*/**

-ROUND

if we have a number 3.516

we use *ROUND(3.516, 2)* this
means we only want to keep 2
decimal places

the 3rd decimal digit will
decide if the 2nd decimal digit
will remain the same or
increase by 1

if the 3rd decimal digit is
greater than 5 then the 3.51
will change to 3.52

and if its smaller then it will
remain the same

**/*

--ROUND - to 2 digit places

select

3.516,

*ROUND(3.516, 2) as
approx*

--ROUND - to 1 digit places

select

3.516,

*ROUND(3.516, 1) as
approx*

--ROUND - to 0 digit places

select

3.516,

*ROUND(3.516, 0) as
approx*

--ABS (absolute)

--this converts negative figures to positive

--if the numbers are already positive nothing gonna happen

--can be used in customers table where due to some calculation mistake the sales is in negative

select

-10,

ABS(-10)

--DATE AND TIME FUNCTIONS

--date format: - yy-MM-DD

--time format: - HH:MM:SS

select

o.OrderID,

o.OrderDate,

o.ShipDate,

o.CreationTime

from Sales.Orders as o

*/**

3 different sources in order to query the dates

I. dates stored inside db (like above)

II. Hardcoded const str value (means writing date on our own after select '2025-10-30')

III. using function GETDATE()

**/*

--GETDATE

--returns the current date and time at the moment when the query is executed

select

GETDATE()

--add a col in orders table having current date and time

select

o.OrderID,

o.OrderDate,

o.ShipDate,

o.CreationTime,

GETDATE() as Today

from Sales.Orders as o

--MANIPULATING DATE AND TIME WITH THE HELP OF SQL FUNCTIONS

*/**

WE CAN DO

1. PART EXTRACTION (like separate year 2025, separate month 08, separate date 25)

2. CHANGING THE FORMAT of the date and separators also used between them (like 08/20/25, 20 Aug 2025, 20.08.2025)

3. CALCULATION (like adding 3 years, subtracting 30days)

**/*

--DATE AND TIME FUNCTIONS

*/**

PART EXTRACTION - Day, Month, Year, Datepart, Datename, Datetrunc, Eomonth

FORMAT AND CASTING - Format, Convert, Casting

CALCULATIONS - Dateadd, Datediff

VALIDATION - Isdate

**/*

--date time PART EXTRACTION**--(DAY, MONTH, YEAR)**

--DAY(date) (returns the day from the date)

--MONTH(date) (returns the month from the date)

--YEAR(date) (returns the year from the date)

--select the year, month, day of creation from the order creation time in separate cols in one table

select

o.OrderID,

o.CreationTime,

YEAR(o.CreationTime) as Year_of_creation,

MONTH(o.CreationTime) as Month_of_creation,

```
DAY(o.CreationTime)
as Day_of_creation,
o.OrderDate
```

```
from Sales.Orders as o
```

--DATEPART(part, date)

--inside the sql (dates also have the week and quarter but not visible(we can extract those using this func))

--we can write full part name also like HOUR and write its abbreviation too like HH and same applies to (year, month, day, hour, minute, second)

--returns a specific part of a data as a number (int)

--extra datepart arguments (Millisecond, Microsecond, nanosecond, isoweek)

--return the year, month, day, hour, minute, second using their abbreviations with function DATEPART

```
select
```

```
o.OrderID,
```

```
o.CreationTime,
```

```
DATEPART(yy,o.CreationTime) as Year_of_creation,
```

```
DATEPART(MM,o.CreationTime) as
Month_of_creation,
```

```
DATEPART(DD,o.CreationTime) as Day_of_creation,
```

```
DATEPART(HH,o.CreationTime) as
Hour_of_creation,
```

```
DATEPART(MM,o.CreationTime) as
Min_of_creation,
```

```
DATEPART(SS,o.CreationTime) as Sec_of_creation,
```

```
o.OrderDate
```

```
from Sales.Orders as o
```

----return the year, month, day, hour, minute, second using function DATEPART

```
select
```

```
o.OrderID,
```

```
o.CreationTime,
```

```
DATEPART(YEAR,o.CreationTime) as
Year_of_creation,
```

```
DATEPART(MONTH,o.CreationTime) as
Month_of_creation,
```

```
DATEPART(DAY,o.CreationTime) as
Day_of_creation,
```

```
DATEPART(HOUR,o.CreationTime) as
Hour_of_creation,
```

```
DATEPART(MINUTE,o.CreationTime) as
Min_of_creation,
```

```
DATEPART(SECOND,o.CreationTime) as
Sec_of_creation,
```

```
o.OrderDate
```

```
from Sales.Orders as o
```

--return the quarter, weekday, week using datepart function

```
select
```

```
OrderID,
```

```
DATEPART(QUARTER,CreationTime) as
Quarter_of_creation,
```

```
DATEPART(WEEK,CreationTime) as
Week_of_creation,
```

```
DATEPART(WEEKDAY,CreationTime) as
Weekday_of_creation
```

```
from Sales.Orders
```

--DATENAME(part, date)

--returns the name of the month or weekday from the date

--returns string in the result

--return the month name, week name of the creation of order

```
select
```

```
OrderID,
```

```
DATENAME(mm,CreationTime) as
Month_of_creation,
```

```
DATENAME(weekday,CreationTime) as
WeekDay_of_creation
```

```
from Sales.Orders
```

--DATETRUNC(part, date)

--truncates the date to the specific part (keeps the specified part and reset remaining datetime)

/*

like 2025-08-20 18:55:45

now when I apply datetrunc to the above date

DEFAULT RESET VALUE
FOR TIME is 00

DEFAULT RESET VALUE
FOR DATE is 01

1. DATETRUNC(year, 2025-08-20 18:55:45) output - 2025-01-01 00:00:00

2. DATETRUNC(month, 2025-08-20 18:55:45) output - 2025-08-01 00:00:00

3. DATETRUNC(day, 2025-08-20 18:55:45) output - 2025-08-20 00:00:00

4. DATETRUNC(hour, 2025-08-20 18:55:45) output - 2025-08-20 18:00:00

5. DATETRUNC(minute, 2025-08-20 18:55:45) output - 2025-08-20 18:55:00

*/

select

OrderID,

DATETRUNC(YEAR, CreationTime) as year_dt,

DATETRUNC(MONTH, CreationTime) as Month_dt,

DATETRUNC(DAY, CreationTime) as Day_dt,

DATETRUNC(HOUR, CreationTime) as Hour_dt,

DATETRUNC(MINUTE, CreationTime) as Min_dt

from Sales.Orders

--actual use of DATETRUNC in data analysis

--grouping of dates while doing analysis

/*

in the creation time they cant be grouped because

every order is not done on the same time all are different time day and month

(so let suppose if I want to group by the results on month of creation)

we can use the datetrunc like below

*/

select

DATETRUNC(MONTH, CreationTime) as Creation,

COUNT(OrderID) as total_orders

from Sales.Orders

group by

DATETRUNC(MONTH, CreationTime)

--EOMONTH(DATE)

--returns the last day of the month

--it will only effect the day of the date (it will change the day to the end day of the month)

--if the day is already on the end day of the month it will remain the same

--return the end day of the month of creation of date

select

OrderID,

DATETRUNC(DAY, CreationTime) as Creation_Date,

EOMONTH(CreationTime) as EOMonth

from Sales.Orders

--return the starting day of the month of creation date

select

OrderID,

DATETRUNC(month, CreationTime) as Start_of_month

from Sales.Orders

--DATE AGGREGATION USE CASES

--return how many orders were placed each year

select

```

YEAR(OrderDate) as
Order_year;

Count(OrderID) as
Orders_Places

from Sales.Orders

group by YEAR(OrderDate)

--return how many orders were
placed each year with month

```

```
select
```

```

DATENAME(MM,
OrderDate) as Order_Month,

Count(OrderID) as
Orders_Places

from Sales.Orders

group by DATENAME(MM,
OrderDate)

```

--PART EXTRACTION USE CASES (DATA FILTERING)

```
--TIP: - filtering data using int
is faster than using str
```

```

--show all orders that are
placed during the month of feb

```

```
select
```

```

DATENAME(MM,
OrderDate) as Order_month,

COUNT(OrderId) as
Orders_Placed

from Sales.Orders

where Month(OrderDate) = 02

group by DATENAME(MM,
OrderDate)

```

```
/*
```

DATATYPE OF THE
RESULTS: -

```

DAY, MONTH, YEAR,
DATEPART = int

DATENAME = str

DATETRUNC = Datetime

EOMONTH = Date

```

```
*/
```

--DATEFORMAT

```
--the default date and time
format is (yy-MM-dd
HH:mm:ss)
```

```
--the sql server follows
INTERNATIONAL STD(ISO
8601) year-month-day
```

--FORMATTING

```
--is the changing of the format
which is set by default
(changing how the data looks
like)
```

```
/*
```

```
DATE FORMATTING: -
```

```
2025-08-20
```

```
MM/dd/yy = 08/20/25
```

```
MMM yy = Aug 2025
```

```
DATE CONVERT: - (have to
provide style number instead of
format)
```

```
CONVERT 6 = 20 Aug 2025
```

```
CONVERT 112 = 20250820
```

NUMBER STYLING /
FORMATTING

```
1234567.89
```

```
FORMAT N = 1,234,567.89
(value will be separated with
comma)
```

```
FORMAT C = $ 1,234,567.89
```

```
FORMAT P = %
123456789.00
```

```
*/
```

--FORMAT AND CASTING

```
--changing the datatype from
one to another
```

```
--can change the datatype with
(cast(), convert())
```

```
/*
```

```
Like if we have a str '123' we
can change it to integer
```

```
if we have a date 2025-08-20
we can change it to string
```

```
if we have a str '2025-08-20'
we can change it to date
```

```
*/
```

--FORMAT(value, format [,culture])

```
--the third parameter is
optional it works by (showing
the value in the style of
specific country or region)
(default culture is en-US)
```

```
--formats a date or time value
```

--for example -
 FORMAT(OrderDate,'dd/MM/yy')

select

OrderID,

CreationTime,

FORMAT(CreationTime,'dd-MM-yy') as
 European_Format,

FORMAT(CreationTime,'MM-dd-yy') as US_Format,

FORMAT(CreationTime,'dd') as dd,

FORMAT(CreationTime,'ddd') as ddd,

FORMAT(CreationTime,'dddd') as dddd,

FORMAT(CreationTime,'MM') as MM,

FORMAT(CreationTime,'MMM') as MMM,

FORMAT(CreationTime,'MMMM') as MMMM

from Sales.Orders

--show creation time using the following format (Day Wed Jan Q1 2025 12:34:56 PM)

--below the datepart can't be used as it returns an int value

--the sql will not be able to cast it to string and it will give an error

--as all the elements are string it will be good to use

DATENAME func instead of DATEQUARTER

select

OrderID,

CreationTime,

'Day ' +

FORMAT(CreationTime,'ddd MMM') + ' Q' +
 DATENAME(QUARTER,CreationTime) + ' ' +

FORMAT(CreationTime,'yy HH:mm:ss tt') as
 Custom_Format

from Sales.Orders

--FORMATTING USE CASES

--DATE AGGREGATIONS

/*the date should be formatted before doing aggregations*/

--show all the orders with their month and year of orders

select

FORMAT(OrderDate,'MMM yy') as OrderDate,

Count(OrderId) as
 total_orders

from Sales.Orders

Group by
 FORMAT(OrderDate,'MMM yy')

--DATA STANDARDIZATION

--FORMAT() - ALL FORMATS (DATE FORMAT SPECIFIERS)

Select

FORMAT(CreationTime,'D') as full_date_Pattern

From Sales.Orders

Similarly as above all the format types stated below can be used on the place of 'D' which shows and formats the date differently.

	FORMAT TYPE
1	D
2	d
3	dd
4	ddd
5	dddd
6	M
7	MM
8	MMM
9	MMMM
10	yy
11	yyyy
12	hh
13	HH
14	m
15	mm
16	s
17	ss
18	f
19	ff
20	fff
21	T
22	t
23	tt

FORMATTED VALUE

1	Monday, December 29, 2025
2	12/29/2025
3	29
4	Mon
5	Monday
6	29-Dec
7	12
8	Dec
9	December
10	25
11	2025
12	6
13	18
14	29-Dec
15	6
16	2025-12-29T18:06:57
17	57
18	Monday, December 29, 2025 6:06 PM
19	54
20	547
21	6:06:57 PM
22	6:06 PM
23	PM

DESCRIPTOIN

1	Full date pattern
2	Short date pattern
3	Day of month with leading zero
4	Abbreviated name of day
5	Full name of day
6	Month without leading zero
7	Month with leading zero
8	Abbreviated name of month
9	Full name of month
10	Two-digit year
11	Four-digit year
12	Hour in 12-hour clock with leading zero
13	Hour in 24-hour clock with leading zero
14	Minutes without leading zero
15	Minutes with leading zero
16	Seconds without leading zero
17	Seconds with leading zero
18	Tenths of a second
19	Hundredths of a second
20	Milliseconds
21	Full AM/PM designator
22	Single character AM/PM designator
23	Two character AM/PM designator

--NUMBER FORMAT SPECIFIERS ALL

SELECT

'N' AS FormatType,
FORMAT(1234.56, 'N') AS
FormattedValue

Similarly as above all the
format types stated below can

be used on the place of 'N'
which formats the numbers
differently.

	FORMAT TYPE	FORMATTED VALUE
1	N	1,234.56
2	P	123456.00%
3	C	\$1,234.56
4	E	1.23E+03
5	F	1234.56
6	N0	1,235
7	N1	1,234.60
8	N2	1,234.56
9	N_de-DE	1.234,56
10	N_en-US	1,234.56

**--CONVERT(datatype ,value
[,style!])**

--it can do both casting and
formatting

--helps in casting a datatype to
another datatype

--give some examples of
converting one datatype to
another

CONVERT(int, '123')[String
to int CONVERT],
select

CONVERT(date, '2025-12-
30')[String to Date
CONVERT],

CONVERT(date,
CreationTime)[Datetime to
date CONVERT],

CONVERT(varchar,
CreationTime, 32)[us std style
: 32],

CONVERT(varchar,
CreationTime, 34)[eu std style
: 34]

from Sales.Orders

--CONVERT ALL STYLES
(CULTURE CODES)

SELECT

'en-US' AS CultureCode,
FORMAT(1234567.89, 'N',
'en-US') AS FormattedNumber,
FORMAT(GETDATE(), 'D',
'en-US') AS FormattedDate

Similarly you can use different
different culture codes to
format the number or date
according to the region
selected

	CULTURE CODE
1	en-US
2	en-GB
3	fr-FR
4	de-DE
5	es-ES
6	zh-CN
7	ja-JP
8	ko-KR
9	pt-BR
10	it-IT
11	nl-NL
12	ru-RU
13	ar-SA
14	el-GR
15	tr-TR
16	he-IL
17	hi-IN

	FORMATTED NO
1	1,234,567.89
2	1,234,567.89
3	1 234 567,89
4	1.234.567,89
5	1.234.567,89
6	1,234,567.89
7	1,234,567.89
8	1,234,567.89
9	1.234.567,89
10	1.234.567,89
11	1.234.567,89
12	1 234 567,89
13	1,234,567.89
14	1.234.567,89
15	1.234.567,89
16	1,234,567.89
17	12,34,567.89

	FORMATTED DATE
1	Monday, December 29, 2025
2	29-Dec-25
3	lundi 29 décembre 2025
4	Montag, 29. Dezember 2025
5	lunes, 29 de diciembre de 2025
6	2025年12月29日
7	2025年12月29日
8	2025년 12월 29일 월요일
9	segunda-feira, 29 de dezembro de 2025
10	lunedì 29 dicembre 2025
11	maandag 29 december 2025
12	29 декабря 2025 г.
13	09/1447/رجب
14	Δευτέρα, 29 Δεκεμβρίου 2025
15	29 Aralık 2025 Pazartesi
16	יום שני 29 דצמבר 2025
17	29 दिसम्बर 2025

--CAST(value as datatype)

--converts a value to a specified data type.

select

CAST('123' as int) [str to int],

CAST(123 as varchar) [int to varchar],

CAST('2025-05-20' as date) [str to date],

CAST('2025-05-20' as datetime) [date to datetime],

CAST(CreationTime as date) [datetime to date]

from Sales.Orders

*/**

CAST - any type to any type, no formatting

CONVERT - any type to any type, formats only date and time

FORMAT - any type to only str, format date, time and numbers

**/*

*/**

--DATEADD DATEDIFF ISDATE

IF YOU USE AN AGGREGATE FUNCTION IN A QUERY THE EVERY OTHER COL IN THE SELECT LIST MUST BE EITHER

INSIDE AN AGGREGATE FUNCTION OR APPEAR IN THE GROUP BY CLAUSE

**/*

--DATEADD(part,interval,date)

--adds or subtracts a specific time interval from or to a date

--increase 2 years, 4 months, and 10 days from every order date

select

OrderID,

OrderDate,

DATEADD(YEAR,2,OrderDate) as two_yrs_later,

DATEADD(MONTH,-4,OrderDate) as before_4months,

DATEADD(DAY,-10,OrderDate) as before_10days

from Sales.Orders

--DATEDIFF(part, start_date, end_date)

--finds the differences between 2 dates

--find the no of days orders took to ship for each month

select

FORMAT(OrderDate, 'MMM yyyy'),

AVG(DATEDIFF(DD, OrderDate, ShipDate)) as avg_delivery_days

from Sales.Orders

Group by
 FORMAT(OrderDate, 'MMM
 yyyy')

--Calculate the age of
 employees

```
select
    EmployeeID,
    FirstName,
    DATEDIFF(yyyy,
    BirthDate, GETDATE()) as
    Age
from Sales.Employees
```

--TIME GAP ANALYSIS

--will use window func (lag()
 in order to access a value from
 the previous value)

--find the no of days between
 each order and the previous
 order

```
select
    OrderID,
    OrderDate as
    Current_Orderdate,
    LAG(OrderDate)
    OVER (Order by OrderDate)
    as Previous_Order_date,--this
    will find the previous order
    date
```

```
DATEDIFF(DAY,
LAG(OrderDate) OVER
(Order by OrderDate),
OrderDate) as NoOfDays
```

from Sales.Orders

--ISDATE(value)

--check if a value is a date

--returns 1 if the string value is
 a valid date 0 if not a valid
 date

```
select
    ISDATE('123') Datecheck1,
    ISDATE('2025-08-20')
    Datecheck2,
    ISDATE('20-08-2025')
    Datecheck3,
    ISDATE('2025') Datecheck4,
    ISDATE('08') Datecheck5;
```

/*Below we can see in the
 subquery that last row contains
 unidentified date format

so it will give error

what we can do is we can
 check if the orderdate is in
 correct format

and set a case statement if the
 isdate output is 1 then only we
 will cast it to date

other wise it will convert to
 2000-01-01*/

```
select
    OrderDate,
    ISDATE(OrderDate)
    Datecheck,
    CASE WHEN
    ISDATE(OrderDate) = 1
    THEN 'VALID'
    ELSE
    'INVALID'
    End Date_Format,
```

```
CASE WHEN
ISDATE(OrderDate) = 1
THEN CAST(OrderDate AS
date)
```

```
ELSE '2000-01-
01'
```

```
END New_order_Date
from
```

--below we just made a
 subquery

/*

SQL Server requires that every
 derived table (a subquery in
 the FROM clause) has an
 alias.

That t is simply the alias name
 you gave to the subquery:

*/

```
(
    select ('2025-08-20') as
    OrderDate UNION
```

```
select ('2025-08-21')
UNION
```

```
select ('2025-08-23')
UNION
```

```
select ('2025-08')
)t
```

```
WHERE ISDATE(OrderDate)
= 0
```

```
--COALESCE, ISNULL, NULLIF,
IS(NOT)NULL
```

--Null

--means nothing! Unknown!
 not equal to anything!

--its not zero! its not empty a string! is not a blank space!

/*

IF WE HAVE A NULL AND
WANT TO REPLACE IT TO A
VALUE

FUNCTIONS AVAILABLE FOR
THAT -> IS NULL, COALESCE

IF WE HAVE A VALUE AND
WANT TO REPLACE IT TO NULL

FUNCTIONS AVAILABLE FOR
THAT -> NULLIF

IF WE WANT TO CHECK IF THE
DATA IS NULL OR NOT

FUNCTIONS AVAILABLE FOR
THAT -> IS NULL, IS NOT NULL

*/

--NULL FUNCTIONS

--ISNULL(value,
replacement_value)

--limited to 2 values

--replaces null with a specified
value

--for ex -

ISNULL(Shipping_Add,
'unknown') --if any of the
shipping address will be null it
will be replaced by unknown

--(Shipping_Add, Billing_add) -
-if the billing add also have
null it will replace the shipping
add value to null

--COALESCE(value1, value 2,
value 3....)

--can have unlimited values

--returns the first non null
value from a list

--coalesce can work the same
as isnull as it can also replace
values

--coalesce(shipping_add,
Billing_Add)

--BUT THE SPECIAL THING IS
THAT WHAT IF THE
REFERENCED TABLE value is
also null at the time of
replacement

--we can fill a static value or
any other value we want

--COALESCE(Shipping_add,
Billing_add, 'N/A')

--first it will check billing_add
if billing_add is null then it will
fillin the static value specified
to Shipping_Add

--ISNULL / COALESCE

--can be used to handle nulls
before doing data aggregations

--sum, count, min, max, avg
totally ignores the null values

--exception if COUNT(*) is
used it will count the null
values too

--find the avg scores for the
customers

--here we didnt replaced the
null values

--nulls are totally ignored

select

CustomerID,

Score,

AVG(Score) OVER()

*AvgScores --after avg (OVER()
AvgScores) is a window func
we will study about it later*

from Sales.Customers;

--replacing nulls and then
finding the avg

--after replacing nulls we will
get different avg

--as nulls are now replaced
with 0

select

CustomerID,

Score,

AVG(COALESCE(Score,

*0)) OVER() AvgScores --
after avg (OVER() AvgScores)
is a window func we will study
about it later*

from Sales.Customers;

--HANDLING NULLS**--mathematical operations**

--display the full name of customers in a single field by merging their first and last names and add 10 points to score each customers

--the coalesce(Score,0) returns the full column of the score after replacing the null values showing replaced and unreplaced value

--below we just added 10 to all the scores

```
select
    CustomerID,
    FirstName,
    LastName,
    FirstName + ' ' +
    COALESCE(LastName, '') as
    FullName,
    Score,
    COALESCE(Score, 0)
+ 10 as Score_with_bonus --
removed 1 null from the last
and added 10 to all of the
scores
from Sales.Customers
```

--HANDLING NULL BEFORE JOINING TABLES

--when joining tables we need the keys to join the tables

--if either of the key is having a null value that record will not show up in the results

--so managing nulls is very important to get all the records

```
/*select
    a.year,
    a.type,
    a.orders,
    b.sales
from table1 a
join table2 b
on a.year = b.year
and ISNULL(a.type, '') =
ISNULL(b.type, '')*/
```

--HANDLING NULL VALUES SORTING DATA

--handle the nulls before sorting data

--if we have nulls the sql goona show it at the start no matter order by desc, or asc

--sort the customers from lowest to highest scores, with nulls appearing last

```
--LAZY WAY
select
    CustomerID,
    Score,
    COALESCE(Score,
99999)
```

from Sales.Customers
order by Score

--PROFESSIONAL WAY

```
select
    CustomerID,
    Score,
    CASE WHEN Score IS
NULL THEN 1 ELSE 0 END
FLAG --made a col flag which
will give 1 if null and 0 if not
null
from Sales.Customers
order by FLAG, Score asc --
after making the flag column
sorting it by it and Score
increasing
```

--NULL IF(value1, value2)

--can only take 2 values

--compares two expressions and returns :

-- Null, if they are equal

-- first value, if they are not equal

--NULL(ORIGINAL_PRICE, DISCOUNTED_PRICE)

/*

Like if we have data like this

Original price = 150,
discounted price = 50 (null if
will return 150)

Original price = 250,
discounted price = 250 (null if
will return null)

*/

--NULL IF usecase

--preventing the error of dividing by zero

--find the sales price for each order by dividing the sales by the quantity

select

OrderID,

Quantity,

Sales,

Sales / Quantity --here this will give error divide by zero error

from Sales.Orders

--logic behind below query

--the null if will check if the quantity is 0 (if any of the quantity is 0 it will be replaced by 0)

select

OrderID,

Quantity,

Sales,

Sales /

NULLIF(Quantity, 0) --here it would have gave error divide by zero error if we haven't applied the nullif function

from Sales.Orders

--IS NULL

--Returns true if the value is null other wise false

--IS NOT NULL

--Returns true if the value is not null otherwise false

--USECASE OF IS NULL

--searching for missing information or nulls

--identify the customers who have no scores

select *

from Sales.Customers

where Score is null

select *

from Sales.Customers

where Score is not null

--ANTI JOIN

--left anti join / right anti join

--finding the unmatched rows between 2 tables

select *

from Sales.Customers

select *

from Sales.Orders

select c.* --the c.* denotes all the cols from the customers table

from Sales.Customers as c

left join Sales.Orders as o

on c.CustomerID = o.CustomerID

where o.CustomerID is null

--NULL VS EMPTY VS BLANKSPACES

--BLANK SPACES

--string value having 1 or more space characters

WITH Orders AS (--this is a cte we will learn about it later in the videos)

select 1 ID, 'A' Category
UNION

select 2, NULL UNION

select 3, " UNION

select 4, ''

)

select *,

DATALLENGTH(Category)
Category_len --a function that counts the length of the data (just like len function)

from Orders;

/*

NULL - (value = null, unknown, Datatype = special marker, storage = very minimal, performance = best, comparison = is null / is not null)

EMPTY STR - (vlaue = "", known empty value, Datatype = string(0), storage = occupies

memory, performance = fast,
comparison = ")

BLANK SPACE - (value = '
'known, space value, Datatype
= string(1 or more), storage =
occupies memory,
performance = slow,
comparison = ' ')

*/

--HANDLING NULLS / DATA POLICIES

--set of rules that how data
should be handled

--DATA POLICY 1

--only use nulls and empty
strings and avoid the blank
spaces

WITH Orders_2 AS (--this is
a cte we will learn about it
later in the videos

SELECT 1 AS ID, 'A' AS
Category

UNION

SELECT 2 AS ID, NULL

UNION

SELECT 3 AS ID, "

UNION

SELECT 4 AS ID, ' '

)

SELECT *,

DATALENGTH(Category)
AS Category_len,

DATALENGTH(TRIM(Category)) as Policy_1

FROM Orders_2;

--DATA POLICY 2

--only use nulls and avoid
using empty strings the blank
spaces

WITH Orders_2 AS (--this is a
cte we will learn about it later
in the videos

SELECT 1 AS ID, 'A' AS
Category

UNION

SELECT 2 AS ID, NULL

UNION

SELECT 3 AS ID, "

UNION

SELECT 4 AS ID, ' '

)

SELECT *,

DATALENGTH(Category)
AS Category_len,

DATALENGTH(TRIM(Category)) as Policy_1,

NULLIF(TRIM(Category),
") as Policy_2 --replaced the
empty strings with nulls

FROM Orders_2;

--DATA POLICY 3

--only use default value
'unkown' and avoid using

nulls, empty strings, and blank
spaces

/*

replacing empty strings and
blanks with NULL during data
preparation before inserting
into a database

to optimize storage and
performance.

*/

WITH Orders_2 AS (--this is a
cte we will learn about it later
in the videos

SELECT 1 AS ID, 'A' AS
Category

UNION

SELECT 2 AS ID, NULL

UNION

SELECT 3 AS ID, "

UNION

SELECT 4 AS ID, ' '

)

SELECT *,

DATALENGTH(Category)
AS Category_len,

DATALENGTH(TRIM(Category)) as Policy_1, --removing the
spaces

NULLIF(TRIM(Category),
") as Policy_2, --replaced the
empty strings with nulls,

COALESCE(nullif(trim(Category),
"), 'unknown') as
Policy_3 --replaced the nulls
with unknown

FROM Orders_2;

--data policy usecases

/*

replacing empty strings and blanks with NULL during data preparation before inserting into a database

to optimize storage and performance.

*/

/*NULLS SUMMARY

special markers means missing value

using nulls can optimize storage and performance

FUNCTIONS

--COALESCE / ISNULL (to replace nulls with a value)

--nullif (to replace a value with a null)

--is null / is not null (to check if the provided data is null or not null)

USECASES

handle nulls - before data aggregation

handle nulls - before mathematical operations

handle nulls - before joining tables

handle nulls - before sorting data

find unmatched data - left anti join

data policies (nulls or default values)

*/

use SalesDB;

--CASE STATEMENT

--evaluates a list of conditions and returns a value when the first condition is met

/*

CASE --start of logic

 WHEN condition1
 THEN result1 --result if condition is true

 WHEN condition2
 THEN result2

 else result --if no condition is true above this will be executed (optional)

END --end of logic

*/

--Main purpose is data transformation

--create new data based on existing columns

--USECASES

--CATEGORIZATION OF DATA - 1

--group the data into different categories based on certain conditions

/*

generate a report showing the total_sales for each category:

High : if the sales is higher than 50

medium : if the sales is between 20 and 50

low : if the sales are equal to 20 or less

sort the result from the highest to lowest

*/

select

Sales_frequency,

SUM(Sales) as Total_Sales

from (--here we used subquery we will learn about this later

select

o.OrderID,

o.Sales,

CASE

 WHEN

o.Sales>50 THEN 'HIGH'

 WHEN

o.Sales>20 AND o.Sales<=50
THEN 'MEDIUM'

 else 'LOW'

end as Sales_frequency

from Sales.Orders as o

)t --the t is the alias without it there will be an error

Group By Sales_frequency

Order By Total_Sales desc

--CASE STATEMENT RULES

--The data type of the results must be matching (like above we used same datatype output for case statements)

--('HIGH', 'MEDIUM', 'LOW')

--USECASE 2 (MAPPING VALUES)

--transform the values from one form to another

--Retrieve employee details with gender displayed as full text

select

EmployeeID,

FirstName,

LastName,

CASE

WHEN Gender
= 'M' THEN 'MALE'

WHEN Gender
= 'F' THEN 'FEMALE'

ELSE 'N/A'

end as Gender_Full

from Sales.Employees

--Retrieve customers details with abbreviated country code

--professional way

select

CustomerID,

FirstName,

Country,

LEFT(Country, 2)

from Sales.Customers

--normal way

select

CustomerID,

FirstName,

Country,

CASE

WHEN Country
= 'Germany' THEN 'GE'

WHEN Country
= 'USA' THEN 'US'

else 'n/a'

end as

Country_abbreviated

from Sales.Customers

--USECASE 3 (QUICK FORM)

/*

1ST WAY is like above which is big and complex and in this we have to write the something multiple times

CASE

WHEN Country
= 'Germany' THEN 'GE'

WHEN Country
= 'USA' THEN 'US'

else 'n/a'

end

*/

--2ND WAY is like below which saves time and helps writing query efficiently

select

CustomerID,

FirstName,

Country,

CASE Country

WHEN
'Germany' THEN 'DE'

WHEN 'USA'
THEN 'US'

ELSE 'N/A'

end as

Country_abbreviated

from Sales.Customers

--USECASE 4 (HANDLING NULLS)

--replace nulls with a specific value

--find the avg scores of customers and treat nulls as 0

```

select
    CustomerID,
    FirstName,
    Score,
    CASE
        WHEN Score is
null THEN 0
        ELSE Score
    end as ScoreClean,
    AVG(CASE --this will
provide proper avg as this
will consider the null as 0 so
it will be counted in the avg
        WHEN
Score is null THEN 0
        ELSE
Score
    end) OVER()
Avg_Cust_Clean,
    AVG(Score) OVER()
Avg_Cust
from Sales.Customers

```

--USECASE 5 (CONDITIONAL AGGREGATION)

--perform aggregation based on certain conditions

--find total sales made in each region but only consider sales greater than 30

```

SELECT
    SUM(CASE
        WHEN
o.Sales > 30 THEN o.Sales
        ELSE 0
    )

```

```

end) as
Total_Sales,
    c.Country
from Sales.Orders as o
LEFT JOIN Sales.Customers
as C
on o.CustomerID =
c.CustomerID
GROUP BY c.Country
--another way to do the same
using CASE statement
select
    SUM(o.Sales) as
Total_Sales,
    c.Country
from Sales.Orders as o
LEFT JOIN Sales.Customers
as C
on o.CustomerID =
c.CustomerID
where o.Sales > 30
GROUP BY c.Country;

```

--count how many times each customer has made an order with sales > than 30

```

SELECT
    CustomerID,
    SUM(CASE WHEN
Sales > 30
        THEN 1
        ELSE 0
    END) as SalesFlag --
made an another col sales

```

flag to help us during the aggregation

--above we have added sales flag col and summed it and grouped by the customer id

```

from Sales.Orders
GROUP BY CustomerID
ORDER BY CustomerID;

```

--count how many times each product has been ordered with sales between 20 and 50

```

select
    ProductID,
    SUM(CASE WHEN
Sales between 20 and 50
THEN 1 ELSE 0 END) as
Sales_Flag
from Sales.Orders
GROUP BY ProductID
ORDER BY ProductID

```

--count how many times each employee has made a sale with quantity > 1

```

SELECT
    e.EmployeeID,
    SUM(CASE WHEN
o.Quantity > 1 THEN 1 ELSE
0 END) AS Qty_flag
from Sales.Employees as e
LEFT JOIN Sales.Orders as o
on e.EmployeeID =
o.CustomerID

```

GROUP BY EmployeeID

ORDER BY Qty_flag

--CASE STATEMENT SUMMARY

--evaluates a list of conditions and returns a value when the first condition is met

/*

RULES

1. The data type of the results must be matching
2. ELSE part is optional

USES

1. Data Categorization
2. Mapping Values
3. Handling NULLs
4. Conditional Aggregation

*/

AGGREGATE FUNCTIONS

use SalesDB;

--COUNT (*) - Counts all rows in a specified table, including those with NULL values.

SELECT

COUNT() AS TotalRows*

from Sales.Customers;

--if you want to count only non-NULL values in a specific

column, use
COUNT(column_name)
instead.

SELECT

COUNT(Score)

from Sales.Customers;

--SUM(column_name) -

Calculates the total sum of a numeric column, ignoring NULL values.

SELECT

SUM(Sales) as TotalSales

from Sales.Orders;

--AVG(column_name) -

Computes the average value of a numeric column, ignoring NULL values.

SELECT

Country,

*AVG(Score) as
AverageScore*

from Sales.Customers

GROUP BY Country

*ORDER BY AverageScore
DESC;*

--MIN(column_name) -

Finds the minimum value in a specified column, ignoring NULL values.

SELECT

*MIN(Sales) as
MinimumSales*

FROM Sales.Orders;

--MAX(column_name) -

Finds the maximum value in a specified column, ignoring NULL values.

SELECT

*MAX(Sales) as
MaximumSales*

from Sales.Orders;

--find the total / avg sales, max and min sales of all order of all orders for all the customers

SELECT

CustomerID,

COUNT() as TotalOrders,*

SUM(Sales) as Total_Sales,

AVG(Sales) as AVG_Orders,

MAX(Sales) as Max_sales,

MIN(Sales) as Min_sales

from Sales.Orders

GROUP BY CustomerID

use SalesDB;

--WINDOW FUNCTIONS

--perform calculations (e.g. aggregation) on a specific subset of data, without losing the level of details of each row

/*

it is very similar to the group by

lets suppose we have 4 orders

order_id | Product | Sales

1	CAPS	10
2	CAPS	30
3	GLOVES	5
4	GLOVES	20

If we want to calculate the total sales for each product earlier we were using GROUP BY:

that group by was combining the same product rows into one row like (2 caps rows become 1 row with total sales 40 and 2 gloves rows become 1 row with total sales 25)

the output will be like this:

Product | Total_Sales

CAPS | 40

GLOVES | 25

IF WE USE WINDOW FUNCTIONS:

to find the total sales for each product without losing the details of each order row we can use window functions (it performs row level calculations)

the output will be like this:

order_id | Product | Sales | Total_Sales

1	CAPS	10	40
---	------	----	----

2	CAPS	30	40
---	------	----	----

3	GLOVES	5	25
---	--------	---	----

4	GLOVES	20	25
---	--------	----	----

so by using the window function we are able to see the total sales for each product while still keeping the details of each order row on the other hand group by would have combined the rows and we would have lost that details

GROUP BY AND WINDOW FUNCTION DIFFERENCE

GROUP BY | WINDOW FUNCTION

simple aggregations | aggregations + details of each row

simple data analysis | advanced data analysis

GROUP BY FUNCTIONS

Aggregate functions : count, sum, avg, min, max

WINDOW FUNCTIONS FUNCTIONS

Aggregate functions [EXCEPT COUNT (only numeric datatype is allowed)] : count(expr), sum(expr), avg(expr), min(expr), max(expr)

Ranking functions [EMPTY] : row_number(), rank(), dense_rank(), cume_dist(), percent_rank(), ntile(number)

Value (analytic) functions [ALL DATA TYPE] : lead(expr, offset, default), lag(expr, offset, default), first_value(expr), last_value(expr), nth_value(expr, n)

*/

--we will

find why group by is not sufficient

and how window functions can help us:

--Find the total sales across all orders

SELECT

SUM(Sales) AS Total_Sales
FROM Sales.Orders;

--Find the total sales for each product

SELECT

ProductID,
SUM(Sales) AS Total_Sales
from Sales.Orders

GROUP BY ProductID

ORDER BY Total_Sales
DESC;

--Find the total sales for each product along order id and Order date

SELECT

OrderID,

ProductID,

OrderDate,

Sales,

SUM(Sales) Total_Sales

from Sales.Orders;

GROUP BY OrderID;

--this query will give error because we are using group by on order id only so we can't select other columns without aggregating them

--now using window functions we can achieve this

SELECT

OrderID,

ProductID,

OrderDate,

SUM(Sales)

OVER(PARTITION BY

ProductID) AS

Total_Sales_by_Product

from Sales.Orders;

/*1. here we will get 10 rows because window functions returns a result for each row

2. we are using OVER() clause to specify that we are using window function

3. PARTITION BY is similar to GROUP BY

4. it divides the result set into partitions to which the window function is applied

5. above query will give total sales for each product along with order id and order date*/

--WINDOW FUNCTIONS SYNTAX

/*

1. window function (any aggregate / ranking / value(analytics functions) function)

1.1 like = AVG(Sales) inside avg there is a function expression (arguments we pass to a function)

2. OVER (partition clause, Order by clause, frame clause)

2.2 all the argument inside over are optional we can leave it empty

--for ex we have the following query

AVG(Sales) OVER(Partition By Category Order by OrderDate)

*/

--PARTITION BY

--divides the result set into partitions to which the window function will be applied

--if we dont use partition by then the entire result set is treated as a single partition

--and calculation is performed on entire data set

--if we apply the PARTITION BY then calculation is done individually on each window(partition)

--Flexibility of window : allows aggregation of data at different granularities within the same query

/*

let suppose we have data of

Month | Product | Sales
(having the month of order, product name and sales)

if we apply (SUM(Sales) OVER()) --> no partition by

then it will give total sales across all months and products

means if we have 10 rows it will return total sales across all rows (entire result set) because we haven't used the PARTITION BY clause

IF WE APPLY SUM(Sales) OVER(PARTITION BY Month)

then it will give total sales for each month

*/

--Find the total sales across all orders additionally provide details order id and order date

SELECT

OrderID,
OrderDate,
SUM(Sales) OVER() as
Total_Sales
from Sales.Orders;

--Find the total sales for each product, additionally provide details such as order id and order date

SELECT
o.OrderID,
p.ProductID,
p.Product,
o.OrderDate,
SUM(o.Sales)
OVER(PARTITION BY
o.ProductID) as Total_Sales
from Sales.Orders as o
LEFT JOIN Sales.Products as
p
on o.ProductID =
p.ProductID;

--Find the total sales for each combination of product and order status

SELECT
o.OrderID,
o.ProductID,
p.Product,
o.OrderStatus,

SUM(o.Sales)
OVER(PARTITION BY
o.ProductID, o.OrderStatus)
as
[TotalSalesByProdct&OrderSt
atus]
from Sales.Orders as o
LEFT JOIN Sales.Products as
p
on o.ProductID = p.ProductID
ORDER BY o.OrderStatus;

**--ORDER BY (SORT THE DATA
WITHIN A WINDOW
ASC/DESC)**

--for the RANK FUNCTIONS
AND VALUE FUNCTIONS
they are must

--rank each order based on
their sales from highest to
lowest additionally provide
order id and order date

SELECT
OrderID,
OrderDate,
Sales,
RANK() OVER (ORDER BY
Sales DESC) RankSales
from Sales.Orders;

--WINDOW FRAME
CLAUSE

--defines a subset of rows
within each window that is
relevant for the calculation

/*

understanding with the help of
an example

1. we have entire data

2. let suppose after putting a
partition by the data is
partitioned in 2 windows

3. now what frame does is
(selects a subset of data from a
window on which the
calculations need to be done
(means window inside a
window))

4. We are here defining scope
of rows (means all the rows in
the windows should not be
included in the calculation
only the chosen rows should
be included)

*/

/*

SYNTAX OF

FRAME CLAUSE

1. AVG(Sales) 2. OVER(3.
PARTITION BY Category 3.
ORDER BY OrderDate 4.
ROWS BETWEEN
CURRENT ROW AND
UNBOUNDED
PRECEEDING)

[] MEANS on the keywords
means they are used above in
the syntax

I.FRAME TYPES = [ROWS],
RANGE (means on the place
of rows the range can also be
used)

II. [BETWEEN]

III. FRAME BOUNDARY
(Lower Value) = [CURRENT
ROW] (accepts 3 types of
keywords : CURRENT ROW,
N PRECEEDING (number of
preceeding), UNBOUNDED
PRECEEDING)

IV. FRAME BOUNDARY
(Higher Value) = accepts 3
types of keywords :
([CURRENT ROW], N
FOLLOWING,
UNBOUNDED
FOLLOWING)

*/

/*

RULES OF FRAME CLAUSE

1. can only be used together
with order by clause
2. Lower Value must be before
the Higher Value

*/

--SUM(Sales)

--Over (ORDER BY Month
ROWS BETWEEN
CURRENT ROW and 2
FOLLOWING)

/*

EXAMPLE-

we have a data

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

WORKING OF THE FRAME CLAUSE QUERY

1. sql gonna process the data
row by row
2. the current row (means the
1st row)
3. we say the range until 2
rows
4. next pointer will be on the
MARCH row because we have
defined 2 following
5. so sql got the scope for the
calculations
6. so it will sum the sales from
january to march
($20+10+30=60$) the result will
be 60 for the january row
7. then sql will move to the
next row (february)
8. now the current row is
february
9. and 2 followign rows
(means from february to april)
10. so it will sum the sales
from february to april
($10+30+5=45$) the result will
be 45 for the february row
11. then sql will move to the
next row (march)

12. now the current row is
march

13. and 2 followign rows
(means from march to june)

14. so it will sum the sales
from march to june
($30+5+70=105$) the result will
be 105 for the march row

15. then sql will move to the
next row (april)

16. now the current row is
april

17. and 2 followign rows
(means april, june, and nothing
(because there are nore more
rows))

18. so it will sum the sales
april, june($5+70=75$) the result
will be 75 for the april row

19. then sql will move to the
next row (june)

20. now the current row is june

21. and 2 followign rows
(means only june (because
there are no rows below june))

22. so it will sum the sales
june(70) the result will be 70
for the june row

*/

--SUM(Sales)

--Over (ORDER BY Month
ROWS BETWEEN
CURRENT ROW and
UNBOUNDED
FOLLOWING)

/*

--UNBOUNDED
FOLLOWING - the last
possible row in the partition

EXAMPLE-

we have a data

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

WORKING OF THE FRAME
CLAUSE QUERY

1. sql gonna process the data
row by row

2. the current row (means the
1st row)

3. we say the range until
UNBOUNDED FOLLOWING
(means until the last row)

4. so sql got the scope for the
calculations

5. so it will sum the sales from
january to june
(20+10+30+5+70=135) the
result will be 135 for the
january row

6. then sql will move to the
next row (february)

7. now the current row is
february

8. and UNBOUNDED
FOLLOWING (means until
the last row)

9. so it will sum the sales from
february to june
(10+30+5+70=115) the result
will be 115 for the february
row

10. then sql will move to the
next row (march)

11. now the current row is
march

12. and UNBOUNDED
FOLLOWING (means until
the last row)

13. so it will sum the sales
from march to june
(30+5+70=105) the result will
be 105 for the march row

14. then sql will move to the
next row (april)

15. now the current row is
april

16. and UNBOUNDED
FOLLOWING (means until
the last row)

17. so it will sum the sales
from april to june (5+70=75)
the result will be 75 for the
april row

18. then sql will move to the
next row (june)

19. now the current row is june

20. and UNBOUNDED
FOLLOWING (means until
the last row)

21. so it will sum the sales
june(70) the result will be 70
for the june row

*/

--SUM(Sales)

--Over (ORDER BY Month
ROWS BETWEEN 1
PRECEDING and CURRENT
ROW)

/*

1 PRECEDING - means the
row before the current row

--means the 1 PRECEDING
will always point to the row
before the current row

EXAMPLE-

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

1. we are on the 1st row
(january)

2. there is no preceding row so
only january sales will be
considered (20)

3. the result will be 20 for
january row

4. then we move to the next
row (february)

5. now we are on february row

6. the preceding row is january
-- means the row before the
february row

7. so sales of january and
february will be considered

(20+10=30) the result will be 30 for february row

8. then we move to the next row (march)

9. now we are on march row

10. the preceding row is february -- means the row before the march row

11. so sales of february and march will be considered (10+30=40) the result will be 40 for march row

10. and then so on.

*/

--SUM(Sales)

--Over (ORDER BY Month ROWS BETWEEN UNBOUNDED PRECEDING and CURRENT ROW)

/*

UNBOUNDED PRECEDING - means the first possible row in the partition/WINDOW

--means the UNBOUNDED PRECEDING will always point to the first row of the partition/window

EXAMPLE-

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

1. we are on the 1st row (january)

2. the UNBOUNDED PRECEDING will point to the january row

3. so only january sales will be considered (20)

4. the result will be 20 for january row

5. then we move to the next row (february)

6. now we are on february row

7. the UNBOUNDED PRECEDING will point to the january row

8. so sales of january and february will be considered (20+10=30) the result will be 30 for february row

9. and then so on.

*/

--SUM(Sales)

--Over (ORDER BY Month ROWS BETWEEN 1 PRECEDING and 1 FOLLOWING)

/*

1 PRECEDING - means the row before the current row

1 FOLLOWING - means the row after the current row

--means the 1 PRECEDING will always point to the row before the current row

--means the 1 FOLLOWING will always point to the row after the current row

EXAMPLE-

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

1. we are on the 1st row (january)

2. there is no preceding row but there is 1 following row so only january and february sales will be considered (20+10=30)

3. the result will be 30 for january row

4. then we move to the next row (february)

5. now we are on february row

6. the preceding row is january -- means the row before the february row

7. the following row is march - means the row after the february row

8. so sales of january, february and march will be considered (20+10+30=60) the result will be 60 for february row

9. then we move to the next row (march)

10. now we are on march row

11. the preceding row is february -- means the row before the march row

12. the following row is april - means the row after the march row

13. so sales of february, march and april will be considered ($10+30+5=45$) the result will be 45 for march row

14. and then so on.

*/

--SUM(Sales)

--Over (ORDER BY Month ROWS BETWEEN UNBOUNDED PRECEDING and UNBOUNDED FOLLOWING)

/*

UNBOUNDED PRECEDING - means the first possible row in the partition/WINDOW

UNBOUNDED FOLLOWING - means the last possible row in the partition/WINDOW

--means the UNBOUNDED PRECEDING will always point to the first row of the partition/window

--means the UNBOUNDED FOLLOWING will always point to the last row of the partition/window

EXAMPLE-

month | sales

january | 20

february | 10

march | 30

april | 5

june | 70

1. starting will be from (unbounded preceeding) january row (the first possible row of the partition/window)

2. ending will be at (unbounded following) june row (the last possible row of the partition/window)

3. so the sum of all the rows will be considered ($20+10+30+5+70=135$)

4. so the result will be 135 for all the rows

*/

SELECT *

from Sales.Orders;

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales)

OVER(PARTITION BY OrderStatus ORDER BY OrderDate ROWS BETWEEN

CURRENT ROW and 2 FOLLOWING) as Total_Sales from Sales.Orders;

/*

OrderID OrderDate
OrderStatus Sales
Total_Sales

1	2025-01-01	Delivered
10	55	
3	2025-01-10	Delivered
20	95	
5	2025-02-01	Delivered
25	105	
6	2025-02-05	Delivered
50	80	
7	2025-02-15	Delivered
30	30	
2	2025-01-05	Shipped
15	165	
4	2025-01-20	Shipped
60	170	
8	2025-02-18	Shipped
90	170	
9	2025-03-10	Shipped
20	80	
10	2025-03-15	Shipped
60	60	

here we can see in the order id 7 that that (the total sales is coming as 30)

because after the order id 7 is another partition or another window (of another orderstatus)

*/

--COMPACT FRAME

--for only preceeding, the syntax of current row can be skipped

/*

NORMAL FORM - ROWS
BETWEEN CURRENT ROW
AND 2 preceeding

SHORT FORM - ROWS 2
PRECEEDING

*/

--EXAMPLE 1

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales)

OVER(PARTITION BY
OrderStatus ORDER BY
OrderDate ROWS 2
PRECEDING) AS
two_preceding

from Sales.Orders;

--EXAMPLE 2

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales)

OVER(PARTITION BY
OrderStatus ORDER BY

OrderDate ROWS 2
PRECEDING) AS
two_preceding

from Sales.Orders;

--DEFAULT FRAME

--is only working with the
order by

--sql uses default frame if
order by is used without frame

--default is (rows between
unbounded preceding and
current row) / (rows
unbounded preceding)

--both are same the 1st is
normal and 2nd is the shortcut
and compact

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales)

OVER(PARTITION BY
OrderStatus ORDER BY
OrderDate) AS two_preceding
from Sales.Orders;

--if we remove the order by

--full window will be
aggregated

SELECT

OrderID,

OrderDate,

OrderStatus,

Sales,

SUM(Sales)

OVER(PARTITION BY
OrderStatus)

from Sales.Orders;

--RULES OF WINDOW FUNCTIONS

/*

1. Wnindow functions can
only be used in the select
clause or the order by clause of
a query

2. cannot be used in WHERE,
GROUP BY, HAVING clauses

3. nested window functions are
not allowed (means we cant
use window function inside
another window function)

4. sql executes the window
functions after the where
clause

5. window function can be
used together with GROUP
BY clause in the same query,
only if the same columns are
used

*/

--Rank the customers based on
their total sales

--use the group by for simpler
aggregations

SELECT

CustomerID,

SUM(Sales) as Total_Sales,

*RANK() OVER(ORDER BY
SUM(Sales)) as Rank*

from Sales.Orders

GROUP BY CustomerID;

--(always be careful while
using window function with
the group by)

--(as long as we are using the
same columns as used with
group by nothing is going to
go wrong and sql will allow)

--SUMMARY (WINDOW FUNCTIONS)

--performs the calculation on
the subset of data without
losing the another important
details

/*

WINDOW FUNCTIONS VS
GROUP BY

1. WINDOW FUNCTION IS
MORE POWER FUL AND
DYNAMIC THAN GROUP
BY

2. DATA ANALYSIS =
ADVANCED WITH
WINDOW FUNCTIONS /
SIMPLE WITH GROUP BY

3. Use GROUP BY +
WINDOW in same query, only
if same column is used

*/

--STEP 1 - TO USE GROUP
BY

--STEP 2 - TO USE
WINDOW FUNCTION

AGGREGATE WINDOW FUNCTIONS

/*

AGGREGATE FUNCTIONS

1.supported datatypes

COUNT(EXPR) - any data
type

SUM(EXPR) - numeric type

AVG(EXPR) - numeric type

MIN(EXPR) - numeric type

MAX(EXPR) - numeric type

2. clauses

COUNT(EXPR) PARTITION
BY (OPT), ORDER BY(OPT),
FRAME CLAUSE(OPT)

SUM(EXPR) PARTITION BY
(OPT), ORDER BY(OPT),
FRAME CLAUSE(OPT)

AVG(EXPR) PARTITION BY
(OPT), ORDER BY(OPT),
FRAME CLAUSE(OPT)

MIN(EXPR) PARTITION BY
(OPT), ORDER BY(OPT),
FRAME CLAUSE(OPT)

MAX(EXPR) PARTITION BY
(OPT), ORDER BY(OPT),
FRAME CLAUSE(OPT)

3.functions work

COUNT(EXPR) - RETURNS
THE NUMBER OF ROWS IN
A WINDOW

SUM(EXPR) - RETURNS
THE SUM OF VALUES IN A
WINDOW

AVG(EXPR) - RETURNS
THE AVG OF VALUES IN A
WINDOW

MIN(EXPR) - RETURNS
SMALLEST VALUE IN A
WINDOW

MAX(EXPR) - RETURNS
BIGGEST VALUE IN A
WINDOW

*/

--COUNT()

--count the total number of
products

SELECT

*COUNT(OrderID) OVER()
as Total_Orders*

from Sales.Orders;

--count the number of orders
each product

SELECT

ProductID,

OrderID,

Sales,

*COUNT(OrderID)
OVER(PARTITION BY
ProductID) as [Number of
Order per Product]*

from Sales.Orders;

--count total number of order
for each customers

--(used for group wise analysis; to understand patterns within different categories)

SELECT

CustomerID,

COUNT(OrderID)
OVER(PARTITION BY
CustomerID)
OrderByCustomers

from Sales.Orders;

--find the total number of customers, also provide all customers details

SELECT

CustomerID,

FirstName,

LastName,

Country,

Score,

COUNT(CustomerID)
OVER() Total_Customers
from Sales.Customers;

--find the total number of scores for the customers

SELECT

CustomerID,

FirstName,

LastName,

Country,

Score,

COUNT(CustomerID)
OVER() Total_Customers,
COUNT(Score) OVER() as
NoOfScores
from Sales.Customers;

USECASE - DATA QUALITY
ISSUES

--leads to inaccuracies in
results

--COUNT() can be used to
identify duplicates

--CHECK whether the table
orders archive contains any
duplicate rows

--IF WE WANT TO FILTER
THE WINDOW FUNCTION

--We cant wrap the query in
SUBQUERY OR
CTE(COMMON TABLE
EXPRESSION)

SELECT

OrderID,

COUNT(OrderID)
OVER(PARTITION BY
OrderID) as Total_Orders
from Sales.OrdersArchive;

--using group by

SELECT

OrderID,

COUNT(OrderID) as
duplicacyCheck
from Sales.OrdersArchive

GROUP BY OrderID;

--using the subquery add a
filter in the query to see only
rows having duplicate data

SELECT

*

FROM(

SELECT

OrderID,

COUNT(OrderID)
OVER(PARTITION BY
OrderID) duplicacyCheck
FROM Sales.OrdersArchive
)t WHERE duplicacyCheck >
1

--COUNT USE CASES

/*

1. Overall Analysis

2. Category Analysis

3. Quality checks : Checking
Nulls

4. Quality check : Identify
Duplicates

*/

--SUM(expr)

--returns the sum of all values
within a window

--Find the total sales across all
orders and total sales for each

product along with all details
orderid, and orderdate

SELECT

OrderID,

ProductID,

SUM(Sales) OVER()

Total_Sales,

SUM(Sales)

OVER(PARTITION BY

ProductID)

Total_Sales_of_Products

from Sales.Orders;

--COMPARISON ANALYSIS
/ AVERAGE ANALYSIS /
PART TO WHOLE
ANALYSIS / COMPARE TO
EXTREME ANALYSIS

--compare the current value
and aggregated value of
window functions

--compare current sales to total
sales

SELECT

OrderID,

ProductID,

SUM(Sales) OVER()

Total_Sales,

SUM(Sales)

OVER(PARTITION BY

ProductID)

Total_Sales_per_Product,

MAX(Sales) OVER()

Max_sale,

MIN(Sales) OVER()

Min_sale,

AVG(Sales) OVER()

Total_Avg,

AVG(Sales)

OVER(PARTITION BY

ProductID) Avg_per_product

from Sales.Orders;

--find the %age contribution of
each product's sales to the total
sales

SELECT *

FROM(

SELECT

OrderID,

ProductID,

Sales,

SUM(Sales) OVER()

Total_Sales,

CONCAT(ROUND((CAST(Sa
les as float) / SUM(Sales)
OVER()) * 100, 2), '%') as
PercentageOfTotal

from Sales.Orders)t

ORDER BY Sales DESC;

--AVG()

--returns the avg of values
within a window

--find the avg sales across all
orders and avg sales for each
product

--and provide details such as
orderid and order date

--GROUP WISE ANALYSIS

SELECT

OrderID,

ProductID,

Sales,

AVG(Sales) OVER() as
Overall_AVG,

AVG(COALESCE(Sales,0))
OVER(PARTITION BY
ProductID) as Avg_of_Sales

from Sales.Orders;

SELECT

CustomerID,

FirstName,

LastName,

AVG(Score) OVER() as
AVGScore,

COALESCE(Score, 0) as
CustomerScores,

AVG(COALESCE(Score,
0)) OVER() as
AVGScoreWithoutNulls

from Sales.Customers;

--Find all orders where sales
are higher than the avg sales
across all orders

--we have used the sub query
(because we cant do it in one
query)

SELECT *

FROM(

```

SELECT
    OrderID,
    ProductID,
    OrderDate,
    Sales,
    AVG(COALESCE(Sales,
0.0)) OVER () AS
avg_sales_all
FROM Sales.Orders
)t
WHERE Sales >
avg_sales_all;

--MIN(), MAX()

--returns the lowest value
within a window

--returns the highest value
within a window

--find the highest sales for
each product

--find the lowest sales for each
product

SELECT
    OrderID,
    ProductID,
    Sales,
    MAX(Sales)
OVER(PARTITION BY
ProductID) as
MaxSalePerProduct,
    MIN(Sales)
OVER(PARTITION BY

```

```

ProductID) as
MinSalePerProduct
from Sales.Orders;

--find the highest and lowest
sales across all orders and the
highest and lowest sales for
each product .

--Additionally provide details
such as orderid, orderdate

SELECT
    OrderID,
    OrderDate,
    ProductID,
    MAX(Sales) OVER() as
MaxSalesAllOrders,
    MIN(Sales) OVER() as
MinSalesAllOrders,
    MAX(Sales)
OVER(PARTITION BY
ProductID) as
MaxSalePerProduct,
    MIN(Sales)
OVER(PARTITION BY
ProductID) as
MinSalePerProduct
from Sales.Orders;

--show the employee who have
the highest salary and the
lowest salary

SELECT *
FROM(SELECT
    *,
    MAX(Salary) OVER() as
HighestSalary,

```

```

MIN(Salary) OVER() as
LowestSalary
    from Sales.Employees)t
WHERE Salary =
HighestSalary OR Salary =
LowestSalary;

--find the deviation of each
sales from the max and min
sales amounts

SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    MAX(Sales) OVER()
HighestSale,
    MAX(Sales) OVER() -
Sales as MaxSalesDeviation ,
    MIN(Sales) OVER()
LowestSale,
    Sales - MIN(Sales) OVER()
as MinSalesDev
from Sales.Orders;

--RUNNING TOTAL AND
ROLLING TOTAL

--they aggregate a squence of
members, and the aggregation
is updated each time a new
member is added

--RUNNING TOTAL -
aggregate all values from the
beginning up to the current

```

point without dropping off older data (very great in order to tracking or budget tracking or we track the current total sales from the target)

--ROLLING / SHIFTING WINDOW TOTAL - aggregate all values within a fixed time window (e.g.30DAY) As new data is added the oldest datapoint will be dropped (we always do focused analysis).

/*

MAIN USE -

1. Tracking current sales with target sales
2. Trend analysis - Providing insights into historical patterns

*/

--RUNNING TOTAL

--THE BELOW QUERY WILL BEHAVE LIKE RUNNING TOTAL

--the default frame is (UNBOUNDED PRECEDING AND CURRENT ROW)

SELECT

```

    OrderID,
    ProductID,
    OrderDate,

    DATENAME(MM,
OrderDate) as MONTH,

    Sales,

    SUM(Sales)
OVER(ORDER BY
```

```

DATENAME(MM,
OrderDate)) as
TotalSaleForMonths

from Sales.Orders;
```

--ROLLING TOTAL

--aggregate values within a fixed window (e.g., last 3 months); as new data is added, the oldest datapoint drops off

--fixed window effect

--summing only the two value above it and itself

--FIRST AGGREGATE BY MONTH, THEN APPLY ROLLING TOTAL

SELECT

```

    OrderID,

    OrderDate,

    DATENAME(MM,
OrderDate) as
Month_of_order,

    Sales,

    SUM(Sales)
OVER(ORDER BY
DATENAME(MM,
OrderDate) ROWS
BETWEEN 2 PRECEDING
AND CURRENT ROW) as
Rolling_Total_Sales

from Sales.Orders;
```

--calculate moving avg of sales for each product over time

SELECT

```

    OrderID,

    ProductID,

    OrderDate,

    Sales,

    AVG(Sales)
OVER(PARTITION BY
ProductID) as
SalesPerProduct,

    AVG(Sales)
OVER(PARTITION BY
ProductID ORDER BY
OrderDate) as MovingAvg,

    SUM(Sales)
OVER(PARTITION BY
ProductID) as SumPerProduct,

    SUM(Sales)
OVER(PARTITION BY
ProductID ORDER BY
OrderDate) as MovingSum

    --default frame =
unbounded preceding and
current row

from Sales.Orders;
```

--calculate the moving avg of sales for each product over time, including only the next order

SELECT

```

    OrderID,

    ProductID,

    OrderDate,

    Sales,

    AVG(Sales)
OVER(PARTITION BY
ProductID ORDER BY
OrderDate ROWS BETWEEN
```

CURRENT ROW AND 1 FOLLOWING) RollingAvg

--default frame = unbounded preceding and current row

from Sales.Orders;

--USECASES OF AGGREGATE FUNCTIONS

--overall total

SELECT

OrderID,

OrderDate,

Sales,

SUM(Sales) OVER()
TotalSales

from Sales.Orders;

--total per groups

SELECT

OrderID,

OrderDate,

Sales,

SUM(Sales)
OVER(PARTITION BY
ProductID)
TotalSalesPerProduct

from Sales.Orders;

--running total

--to see progress overtime

SELECT

OrderID,

OrderDate,

Sales,

SUM(Sales)
OVER(ORDER BY
OrderDate)
TotalSalesPerProduct

from Sales.Orders;

--rolling total

--to see progress over time in a
specific timeframe or in a
specific fixed window

SELECT

OrderID,

OrderDate,

Sales,

SUM(Sales)
OVER(ORDER BY OrderDate
ROWS BETWEEN 2
PRECEDING AND
CURRENT ROW)
TotalSalesPerProduct

from Sales.Orders;

--AGGREGATE WINDOW
FUNCTIONS SUMMARY

/*

Aggregates set of values and
return a single aggregated
value

RULES :

1. Except count() all functions
only except numeric
arguments

2. All clauses are optional in
the over clause (like partition
by, order by, frame clause)

USE CASES :

1. Overall Analysis

2. Total Per Group Analysis

3. Part to whole analysis

4. Comparison analysis =
(AVG), (EXTREME : Highest
/ Lowest)

5. Identify Duplicates

6. Identify data quality issues

7. Outlier Detection

8. Running Total

9. Rolling Total

10. Moving AVG

*/

--RANKING FUNCTIONS

/*

In the first step the sql sorts in
desc before data ranking

TYPES OF RANKING
FUNCTIONS

1. INTEGER BASED

RANKING - gives integer
values as rank based on the
values (top/bottom n analysis)
[row_number(), rank(),
dense_rank(), ntile(number)]

2. PERCENTAGE BASED RANKING - gives percentage values as rank based on the values (from the scale of 0 to 1) (distribution analysis)
[cume_dist(), percent_rank()]

Rank functions must be empty (they don't accept any arguments) except for ntile()

PARTITION BY clause is optional

ORDER BY clause is mandatory

Frame clause is not allowed
*/

--ROW_NUMBER() - gives unique number as rank to each row / and it doesn't consider ties (means if two values are same it will give different ranks)

```
SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    ROW_NUMBER()
OVER(ORDER BY Sales
DESC) as SalesRank
from Sales.Orders;
```

--RANK() - gives number as rank to each row / and it considers ties (means if two

values are same it can have same ranks) it leaves gaps in ranking

--means if the 2nd and 3rd value are same and both got the rank 2 then the 4th value will get the rank as 4 rather than getting 3

```
SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    ROW_NUMBER()
OVER(ORDER BY Sales
DESC) as SalesRank
from Sales.Orders;
```

--DENSE_RANK() - assigns rank to each row / it considers ties (means if two values are same it can have same ranks) but it doesn't leave gaps in ranking

```
SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    DENSE_RANK()
OVER(ORDER BY Sales
DESC) as SalesRank
from Sales.Orders;
```

/(row_number() = unique rank without gaps and doesn't consider ties,

rank() = shared rank with gaps & handles ties,

dense_rank() = shared rank without gaps & handles ties*/

--ROW_NUMBER()

--use cases - TOP N ANALYSIS

--find the top highest sales for each product

```
SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    ROW_NUMBER()
OVER(PARTITION BY
ProductID ORDER BY Sales
desc) as Rank_by_Prod
from Sales.Orders;
```

--now to filter above data we have to use subqueries

SELECT *

```
FROM(
    SELECT
        OrderID,
        OrderDate,
        ProductID,
        Sales,
```

```

    ROW_NUMBER()
OVER(PARTITION BY
ProductID ORDER BY Sales
desc) as Rank_by_Prod

    from Sales.Orders)t

WHERE Rank_by_Prod = 1;

--BOTTOM N ANALYSIS

--find the lowest 2 customers
based on their total sales

--with more details such as
fullname, country name

--to filter we are using the
subquery

SELECT

    t.CustomerID,

    CONCAT(c.FirstName, ' '
,c.LastName, '-', c.Country) as
FullNameWithCountry,

    t.Total_Sales,

    t.Rank_by_cust

FROM(

SELECT

    CustomerID,

    SUM(Sales) as Total_Sales,

    ROW_NUMBER()
OVER(ORDER BY
SUM(Sales) ASC) as
Rank_by_cust

from Sales.Orders

GROUP BY CustomerID)t

INNER JOIN Sales.Customers
as c

on t.CustomerID =
c.CustomerID

```

```

WHERE t.Rank_by_cust <= 2;

--GENERATE UNIQUE IDs

--these unique id's can be used
as an identifier

-- and used for paginating
(process of breaking down a
large data set into smaller
manageable chunks)

--assign unique IDs to the rows
of the ordersarchive table

SELECT

    OrderID,

    OrderDate,

    CustomerID,

    Sales,

    ROW_NUMBER()
OVER(ORDER BY
OrderID,OrderDate) as
Unique_ID

from Sales.OrdersArchive;

--IDENTIFY DUPLICATES

--identify duplicates rows in
the table ordersarchive and
return a clean result without
duplicates

/*

PARTITION BY OrderID -
Divides the data into groups by
OrderID. The row numbering
resets to 1 for each new
OrderID group

ORDER BY CreationTime -
Within each OrderID partition,
rows are numbered in the order
of CreationTime (earliest first)

```

```

*/

SELECT *

FROM (

SELECT

    OrderID,

    ProductID,

    OrderDate,

    CustomerID,

    ROW_NUMBER()
OVER(PARTITION BY
OrderID ORDER BY
CreationTime) as rn

from Sales.OrdersArchive)t

where rn=1;

--to see any duplicate data

SELECT *

FROM (

SELECT

    OrderID,

    ProductID,

    OrderDate,

    CustomerID,

    ROW_NUMBER()
OVER(PARTITION BY
OrderID ORDER BY
CreationTime) as rn

from Sales.OrdersArchive)t

where rn>1;

--SUMMARY
(ROW_NUMBER())
USECASES

/*

```

TOP N ANALYSIS

BOTTOM N ANALYSIS

ASSIGN UNIQUE IDS for
duplicacy check

QUALITY CHECKS: Identify
Duplicates

*/

--NTILE(number_of_buckets)

--divides the rows into a
specified number of
approximately equal groups
(Buckets)

--we have to define a
numerical value inside the
ntile

--Bucket size = Number of row
/ Number of buckets

/*

NTILE(2)

LIKE WE HAVE A DATA

SALES | NTILE

100 | 1

80 | 1

80 | 2

50 | 2

the bucket size is calculated
with the help of total number
of rows / number of buckets

$4/2 = 2$ (means each bucket
will have 2 rows)

WE CAN SEE THAT THE
DATA IS DIVIDED INTO 2
BUCKETS

where the first 2 rows are in
bucket 1 and the next 2 rows
are in bucket 2

If there are odd number of
rows then the extra row will be
added to the earlier buckets

NTILE(2)

LIKE WE HAVE A DATA

SALES | NTILE

100 | 1

80 | 1

80 | 1

50 | 2

30 | 2

so above we have 5 rows and 2
buckets

sql rule = larger groups come
first

*/

--NTILE()

SELECT

OrderID,

Sales,

NTILE(1) OVER(ORDER
BY Sales DESC) as Bucket1,

NTILE(2) OVER(ORDER
BY Sales DESC) as Bucket2,

NTILE(3) OVER(ORDER
BY Sales DESC) as Bucket3
from Sales.Orders;

--NTILE() usecase

--will do data segmentation as
a data analyst

--equalizing load processing
and etl as a data engineer

--DATA SEGMENTATION

--divides a dataset into distinct
subsets based on certain
criteria.

--for ex - like grouping up the
customers based on their
behaviour

--(we can segment customer
with their sales, employees
with their salaries, products by
prices and so on)

--segment all orders into 3
categories high, medium and
low sales

SELECT

OrderID,

ProductID,

Sales,

CASE Sales_Rank

WHEN 1 THEN 'HIGH'

WHEN 2 THEN 'MID'

WHEN 3 THEN 'LOW'

END as Sales_Rank

FROM(SELECT

```

    OrderID,
    ProductID,
    Sales,
    NTILE(3)
OVER(ORDER BY Sales
DESC) as Sales_Rank
from Sales.Orders)t

--EQUALIZING LOAD
--load balancing

/*

When moving large table from
one db to another db

if we move the full table in one
go which is large it may take a
long time and we can get
errors too

so we can transfer the table to
another db in parts to make
sure we dont get errors

after transferring we can use the
union to see the full table

*/

--in order to export the data
devide the orders table into
two groups

SELECT *,

    NTILE(2) OVER(ORDER
BY OrderID ASC) as
Table_Group
from Sales.Orders;

--%AGE BASED RANKING

```

```

--cume_dist(), percent_rank()

--these functions are amazing
in order to do distribution
analysis

--CUME_DIST()

--cumulative distribution
calculates the distribution of
data point within a window

--CUME_DIST = position
number of the value / number
of rows

--tie rule = the position of the
last occurence of the same
value will be considered as the
position number of the value

--Inclusive (the current row is
included)

--range [0, 1] never 0

SELECT

    OrderID,
    ProductID,
    Sales,
    CUME_DIST()
OVER(ORDER BY Sales
DESC)

FROM Sales.Orders;

--find the product that falls
within the highest 40% of
prices

SELECT

```

```

ProductID,
Product,
Category,
Price,
Price_Rank,
    CONCAT(Price_Rank *
100, '%') as Dist_Rank
FROM(SELECT

    ProductID,
    Product,
    Category,
    Price,
    CUME_DIST()
OVER(ORDER BY PRICE
DESC) as Price_Rank
FROM Sales.Products)t
WHERE Price_Rank<=0.4;

--PERCENT_RANK()

--calculates the relative
position of each row within a
window

--PERCENT_RANK =
position number of the value -
1 / number of rows - 1

--tie rule = the position of the
first occurence of the same
value will be considered as the
position number of the value

--Exclusive (the current row is
exclusive)

--range [0, 1] can be 0

```

SELECT

OrderID,

ProductID,

Sales,

ROUND(PERCENT_RANK()
OVER(ORDER BY Sales
DESC), 2)as Rank_Percentage

FROM Sales.Orders;

--find the product that falls
within the highest 40% of
prices

SELECT

ProductID,

Product,

Category,

Price,

Price_Rank,

CONCAT(Price_Rank *
100, '%') as Dist_Rank

FROM(SELECT

ProductID,

Product,

Category,

Price,

PERCENT_RANK()
OVER(ORDER BY PRICE
DESC) as Price_Rank

FROM Sales.Products)t

WHERE Price_Rank<=0.4;

--RANK WINDOW
FUNCTIONS SUMMARY

/*

Assign a rank for each row
within a window

Types

INTEGER BASED
RANKING

1. ROW_NUMBER()
2. RANK()
3. DENSE_RANK()
4. NTILE()

PERCENTAGE BASED
RANKING

1. CUME_DIST()
2. PERCENT_RANK()

RULES

1. expression should be empty
(arguments not allowed)
2. we must use order by
3. frame clause is not allowed

USECASES

1. top n alaysis / bottom n
alaysis
2. Identify and remove
duplicates
3. find data quality issues

4. generate unique id's (if our
table dont have clean primary
key) + paginating

5. data segmentation

6. data distribution analysis

7. Equalizing load processing

*/

--VALUE / ANALYTICAL WINDOW FUNCTIONS

--these functions are used to
access value from another row

/*

Lets suppose we have a table:

Month | Sales

jan | 100

feb | 200

mar | 300

apr | 400

may | 500

jun | 600

jul | 700

if we are on the current row
(let suppose on april row)

WE CAN USE

1. LAG() fuction to access
value from previous row
(means mar row)
2. LEAD() function to access
value from next row (means
may row)

3. first_value() function to access value from first row (means jan row)

4. last_value() function to access value from last row (means jul row)

RULES

1. LAG(expr, OFFSET, default) [Expression : All data type, OFFSET : Number of rows forward or backward from current row default = 1, Default : Returns a default value if next or prev row is not available (default = null), Partition Clause : Optional, Order Clause : Required, Frame Clause : Not Allowed]

2. LEAD(expr, OFFSET, default) [Expression : All data type, OFFSET : Number of rows forward or backward from current row default = 1, Default : Returns a default value if next or prev row is not available (default = null), Partition Clause : Optional, Order Clause : Required, Frame Clause : Not Allowed]

3. first_value(expr) [Expression : All data type, Partition Clause : Optional, Order Clause : Required, Frame Clause : Optional]

4. last_value(expr) [Expression : All data type, Partition Clause : Optional, Order Clause : Required, Frame Clause : Should be used]

(so that we can compare the sales of different months)

ORDER BY is mandatory to use

*/

--VALUE WINDOW
FUNCTIONS

--MIN / MAX

--LEAD() Access the value from the next row within a window

--LAG() Access the value from the previous row within a window

--USECASES - 1

--TIME SERIES ANALYSIS (analyzing the data to understand patterns trends and behaviours over time [Business's performance over time]) YoY - Year over Year, MoM - Month over Month

--analyze the month over month (MOM) performance by finding the percentage change in sales between the current and previous month

SELECT *,
Total_Sales -
Previous_Month_Sale as
Sale_Difference,

CONCAT(ROUND(CAST((Total_Sales -

Previous_Month_Sale) AS
FLOAT) /

Previous_Month_Sale * 100,
2), '%') as MoM_Percentage

FROM(SELECT

MONTH(OrderDate) as
OrderMonth,

SUM(Sales) as Total_Sales,

LAG(SUM(Sales)) OVER
(ORDER BY
MONTH(OrderDate)) as
Previous_Month_Sale

from Sales.Orders

GROUP BY
MONTH(OrderDate))t

--USECASE - 2

--Customer Retention Analysis (Measuring customer behaviour and loyalty to help business build strong relationships with customers)

--analyze customer loyalty by ranking customers based on avg no of days between orders

SELECT

CustomerID,

AVG(Days_Interval) as
avginterval,

RANK() OVER(ORDER
BY
COALESCE(AVG(Days_Inter

val), 99999999)) as
Rank_of_Customer

FROM(

SELECT

OrderID,

CustomerID,

OrderDate,

LEAD(OrderDate)
OVER(PARTITION BY
CustomerID ORDER BY
OrderDate) as Lead_order,

DATEDIFF(DAY,
OrderDate, LEAD(OrderDate)
OVER(PARTITION BY
CustomerID ORDER BY
OrderDate)) AS Days_Interval

FROM Sales.Orders)t

GROUP BY CustomerID;

--FIRST_VALUE() - Access a
vlaue from the first row within
a window

--LAST_VALUE() - Access a
value form the last row within
a window

--FIND THE FIRST AND
THE LAST VALUE OF THE
Sales

--the below query of last value
will not be working as
expected because we haven't
used the frame clause in that

SELECT

DATENAME(MM,
OrderDate) as MonthName,

Sales,

FIRST_VALUE(Sales)
OVER(ORDER BY
MONTH(OrderDate)) as
FirstVal,

LAST_VALUE(Sales)
OVER(ORDER BY
MONTH(OrderDate)) as
LastVal

FROM Sales.Orders;

--so to get the expected results
instead do like this

SELECT

DATENAME(MM,
OrderDate) as MonthName,

Sales,

LAST_VALUE(Sales)
OVER(ORDER BY
MONTH(OrderDate) ROWS
BETWEEN CURRENT ROW
AND UNBOUNDED
FOLLOWING) as LastVal

FROM Sales.Orders;

--find the highest and lowest
sales by product

SELECT

OrderID,

ProductID,

Sales,

FIRST_VALUE(Sales)
OVER (PARTITION BY
ProductID ORDER BY Sales)
as LowestSale,

LAST_VALUE(Sales)
OVER (PARTITION BY
ProductID ORDER BY Sales
ROWS BETWEEN
CURRENT ROW AND
UNBOUNDED
FOLLOWING) as
HighestSale,

MIN(Sales)
OVER(PARTITION BY
ProductID) as MinSale,

MAX(Sales)
OVER(PARTITION BY
ProductID) as MaxSale

from Sales.Orders;

/*first_value can also be used
as last value (just by changing
the order by to descending)

SO THESE FUNCTIONS
CAN BE USED TO
COMPARE HOW WELL A
VALUE IS PERFORMING
AS COMPARED TO THE
EXTREMES*/

--SUMMARY WINDOW
VALUE FUNCTIONS /
ANALYTICAL FUNCTIONS

/*

Allow access of specific value
from another row

1. LAG() - PREVIOUS
VALUE

2. LEAD() - NEXT VALUE

3. FIRST_VALUE() - FIRST
VALUE WITHIN A
WINDOW

4. LAST_VALUE() - LAST VALUE WITHIN A WINDOW

RULES

1. Expression - any data type
2. OrderBy - Required
3. Frame - Optional (mostly leave it empty except the LAST_VALUE() function)

USECASES

1. Time series analysis (MoM, YoY)
2. Time Gap analysis (Customer retention)
3. Comparison Analysis : Extreme (HIGHEST, LOWEST)

*/

--IN SQL WE HAVE 5 DIFFERENT TECHNIQUES TO REDUCE AND OPTIMIZE THE COMPLEXITY OF SQL QUERIES

--THESE ARE

--SUBQUERY, CTE(COMMON TABLE EXPRESSION), VIEW, TMP, CTAS(Create Table As Select)

--Question arises - Why do we need those techniques

--SQL DATA PROJECTS (REAL WORLD SCENARIOS)

/*

Normally in projects we have a database and person responsible for the database structure (dba (data base administrator))

In simple scenario we are having a user who is writing queries to retrieve the data from the database and after writing the query the user will be getting the results

BUT IN THE REAL PROJECT THINGS GET VERY COMPLICATED

Let suppose we have

1. Financial Analyst : (that is writing huge block of sql queries which is very complex)
2. Risk Manager : (is also writing complex and huge block of sql queries)
3. and for different departments and different projects for different task we will be having lot of analysts that are writing many complex queries

All of those will be having direct access to the database and they are executing complex sql queries in order to generate report or something

We will also be having DATA ENGINEER

who will be :

1. building data warehouse
2. extract the data
3. writing extract queries
4. after extraction writing different scripts for transformation (in order to manipulate, filter, cleanup, aggregate your data)
5. 3rd script to collect the results of the transformation and load it in another database called data warehouse

DATA WAREHOUSE : A special database that collects and integrates data from different sources to enable analytics and support decision making.

In the end of this chain we will be having a DATA ANALYST

Who will be :

1. writing queries to analyse the data in the data warehouse

Or we might have a different query to prepare the data before insert it into a tool like power bi in order to generate reports and visualization

SO WE CALL THIS A DATA WAREHOUSE SYSTEM / BUSINESS INTELLIGENCE SYSTEM

Now we also have DATA SCIENTIST having direct access to database

Who will be :

1. writing different queries to extract the data
2. manipulating the data that are needed in order to develop a model and doing machine learning and AI

The results of the data analyst gonna be used in another query in order to prepare the results for power bi or exporting the results as and excel list

So we have a lot of people having different roles that want to access the database and do analysis on top of it

because everyone wants to answer the questions based on the data

IN THIS WE WILL FIND MANY CHALLENGES AND PROBLEMS:

1. Redundancy (because if we compare all queries used by everyone we will find we are having same logic repeating over and over)
2. Performance Issues

3. Complexity
4. Hard to Maintain
5. Database stress
6. Security

SOLUTIONS

1. Subqueries
2. CTE
3. Views
4. Temporary Tables
5. CTAS (Create Table As Select)
- */

--SIMPLIFIED DATA ARCHITECTURE

/*

DATABASE ENGINE : Brain of the database handling multiple operations such as storing, retrieving and managing the data within the database.

In the database we have a very important component that is the storage

Two types of storage in a db are :

1. DiskStorage : Long Term memory where the data is stored permanently even if we turn off the system (I can store lot of data, but its slow to read and write)

2. Cache : Short Term memory where data is stored temporarily (Extremely fast to read and write, but can hold only smaller amount of data (for short term only))

DISK STORAGE

is having 3 types

1. USER : Main content of the db. This is where the actual data that users care about is stored
2. TEMP STORAGE : It is a temporary space used by the database for short term tasks, like processing queries and sorting the data. Once these tasks are done, the storage is cleared.
3. CATALOG : DB's internal storage for its own information. A blueprint that keeps track of everything about the database itself, not the user data (it holds the metadata information about the DB [METADATA : Data about Data])

METADATA LOOKS LIKE THIS:

Let suppose we have a customer table its metadata will look like this:

Table Name | Column Name |
Data Type | Size | Constraints

Customer | CustomerID |
INT | 4 | PRIMARY
KEY

```
Customer | FirstName |
VARCHAR | 50 | NOT
NULL
```

```
Customer | LastName |
VARCHAR | 50 | NOT
NULL
```

```
Customer | Email |
VARCHAR | 100 | UNIQUE
```

since we are working on our database we have full access to everything inside the sql server but in real projects if we are just a user or developer we will not have the access to the system databases (only for db administrators)

SO THESE ARE ALL THE COMPONENTS OF THE DATABASE ARCHITECTURE

```
*/
```

```
/*
```

We can find system catalog / metadata in the information schema of the database

INFORMATION SCHEMA :
A system defined schema with built-in views that provide information about all the database objects such as tables, columns, data types, and constraints.

```
*/
```

```
--LIKE
```

```
SELECT
```

```
*
```

```
FROM
INFORMATION_SCHEMA.C
OLUMNS;
```

```
--AND THIS
```

```
SELECT
```

```
DISTINCT TABLE_NAME
```

```
FROM
```

```
INFORMATION_SCHEMA.C
OLUMNS;
```

--SUBQUERY

```
--Query inside another query is
called subquery
```

```
--the subquery supports the
main query with data
```

```
--the subquery can only be
used from the main QUERY
```

```
/*
```

```
step 1 - Aggregations -
(subquery)
```

```
step 2 - Transformations -
(subquery)
```

```
step 3 - Filtering - (subquery)
```

```
step 4 - Join Tables - (MAIN
Query)
```

```
We will be having different
queries for every step
```

```
*/
```

```
--SUBQUERY CATEGORIES
```

```
--WHEN TO USE
SUBQUERIES
```

```
/*
```

DEPENDENCY BETWEEN THE SUBQUERY AND THE MAIN QUERY

Mainly 2 types of subqueries

[Non corelated subquery
(independent from the main query)]

, Corelated subquery
(unindependent from the main query)]

HOW TO GROUP UP THE SUBQUERIES DEPENDING ON THE RESULT TYPE

Scalar Subquery - it returns only 1 single value

Row Subquery - return multiple rows

Table Subquery - returns multiple rows and cols

LAST WAY TO CATEGORIZE SUBQUERY

(Based on the location and clauses)

We are describing here where the subquery gonna be used within the main query

```
Select
```

```
From
```

```
Join
```

Where - (can be used with -
(Comparison Operators),
(Logical Operators (in, any, all, EXISTS)))

```
*/
```

--SUBQUERY (RESULT TYPES)

--1. Scalar subquery - only 1 single value

SELECT

SUM(Sales) as SumOfSales
FROM Sales.Orders;

--the result of above will be a single value we call that value a scalar value

--2. Row subquery - Returns multiple rows as output

SELECT

CustomerID
FROM Sales.Orders;

--the result of above will be a full row which we call Row Query

--3. Table subquery - Returns multiple cols and rows as output

SELECT

*

FROM Sales.Orders;

--the result of above will be a full table (having multiple cols and rows)

--SUBQUERY (From Clause)

--Used as temporary table for the main query

/*

example of subquery -

SELECT col1, col2

FROM(SELECT col FROM table1 WHERE condition)as alias

*/

--Find the products that have a price higher than the avg price of all products

--main query

SELECT

ProductID,

ProductID,

AVG_Price,

Price,

Price - AVG_Price as
PriceDeviation

FROM(SELECT--subquery

ProductID,

Product,

Price,

AVG(Price) OVER()
AVG_Price

FROM Sales.Products)t

WHERE Price>AVG_Price;

--Rank the customers based on their total amount of sales

SELECT

*,

DENSE_RANK()
OVER(ORDER BY TotalSales
DESC) as Rank

FROM

(SELECT

CustomerID,

SUM(Sales) as TotalSales

FROM Sales.Orders

GROUP BY CustomerID)t;

/*

How the query is executed above:

1. Subquery - the subquery is executed first

(the subquery fetches all the data we want and groups the rows by customers)

2. MainQuery - the main query is executed last

(the main query is executed at last and the selected results from the subquery are taken and ranked based on their total sales)

*/

--SUBQUERY (SELECT)

--used to aggregate data side by side with the main query's data, allowing direct comparison

--only scalar subqueries are allowed to be used (the result

must be a single value, We can understand why by seeing the example below)

--show the prod ids, prod names, prices, and the total number of orders

```
SELECT
    ProductID,
    Product,
    Price,
    (SELECT
        COUNT(OrderID)
    FROM Sales.Orders) as
    TotalOrders
FROM Sales.products;
```

--SUBQUERY (JOIN)

--used to prepare the data (filtering or aggregation)

--to dynamically create a result sets for joining with another table

--show all the customer details and find the total orders of each customer

```
SELECT
    c.*,
    o.CountOfOrders
FROM Sales.Customers as c
LEFT JOIN(
    SELECT
```

```
    CustomerID,
    COUNT(OrderID) as
    CountOfOrders
    from Sales.Orders
    GROUP BY CustomerID)
as o
on c.CustomerID =
o.CustomerID
```

--SUBQUERY (WHERE)

--used for complex filtering logic and makes query more flexible and dynamic

--we can use COMPARISON, LOGICAL OPERATORS

--only scalar subquery is allowed ONLY

--the subquery should only be returning single value

```
/*
SELECT col1, col2
FROM Table
WHERE col = (SELECT col
FROM table2 where
condition)
*/
--subquery comparison
operator
--find the products that have a
price higher than the avg price
of all products
SELECT
    ProductID,
    Product,
```

```
Price,
    (SELECT AVG(Price)
FROM Sales.Products) as
AVGPrice
```

from Sales.Products

```
WHERE Price > (SELECT
AVG(Price) FROM
Sales.Products);
```

--subquery logical operators

--subquery in operator

--used to check whether a value matches any value from a list.

--show the details of orders made by customers in Germany

```
SELECT
*
FROM Sales.Orders
```

--subquery any operator

--checks if a value matches any value within a list.

--used to check if a val is true for at least 1 of the values in the list

--find female employees whose salaries are greater than the salaries of any male employee

```
SELECT
*
```

```
FROM Sales.Employees

WHERE CustomerID in
(SELECT CustomerID FROM
Sales.Customers WHERE
COUNTRY = 'Germany');

WHERE Gender = 'F' and
Salary > ANY(SELECT Salary
FROM Sales.Employees
WHERE Gender = 'M');
```

--subquery all operator

--check if a value matches
ALL values within a list

--find female employees
whose salaries are greater than
the salaries of all male
employees

```
SELECT
```

```
*
```

```
FROM Sales.Employees
```

```
WHERE Gender = 'F' and
Salary > ALL(SELECT Salary
FROM Sales.employees
WHERE Gender = 'M');
```

--SUBQUERY (CO
RELATED / NON CO
RELATED)

--before this all the subqueries
we learnt were non co related

--NON CO RELATED
SUBQUERY - a subquery that
can run independently from
the main query

--CO RELATED SUBQUERY

- a subquery that depends on
the main query for its values

```
/*
```

- Dependency on outer query:
The inner query references
columns from the outer query,
making them “correlated.”

- Row-by-row execution:
Unlike a regular subquery
(which runs once
independently), a correlated
subquery runs repeatedly—
once for each row processed
by the outer query.

- Dynamic evaluation: Results
change depending on the
current row being evaluated in
the outer query.

- Common use cases: Filtering,
comparisons, conditional
updates, and ranking.

```
*/
```

--show all customer details and
find the total orders for each
customer

--with correlated subquery

```
SELECT
```

```
*,
```

```
(SELECT COUNT(OrderID)
from Sales.Orders as o
WHERE c.CustomerID =
o.CustomerID) TotalOrders
from Sales.Customers as c;
```

--with join and subquery (old
way)

```
SELECT
```

```
c.*,
```

```
o.TotalOrders
```

```
from Sales.Customers as c
```

```
left JOIN
```

```
(SELECT
```

```
CustomerID,
```

```
COUNT(OrderID) as
TotalOrders
```

```
from Sales.Orders
```

```
GROUP BY CustomerID) o
```

```
on c.CustomerID =
```

```
o.CustomerID;
```

```
/*
```

```
CoRelated
```

```
NonCoRelated
```

```
-----
-----
-
```

subquery is independent
| subquery is dependent on
main query from main query

subquery runs once |
subquery runs multiple times
(for each row of main query

can be executed alone |
cannot be executed alone

easier to read and
understand | harder to read and
understand

executed only once leads to |
executed multiple times leads
to poor performance

better performance

static comparisons,
filtering|Rows by row
comparisons, Dynamic
filtering

with constants

*/

--subquery EXISTS operator

--check if a subquery returns
any rows

--co related subquery in where
clause in where clause exists
operator

/*

SELECT col1, col2

from table

where EXISTS(select 1

from table2

where table.id = table2.id)

here we are just checking if the
subquery is return atleast 1
row

no othe

*/

--show the details of orders
made by customers in germany

SELECT

*

FROM Sales.Orders as o

WHERE EXISTS (SELECT

*

FROM Sales.Customers as c

where Country = 'Germany'

and o.CustomerID =

c.CustomerID);

--above in the subquery no
matter what we are writing in
the select list (it is ignored by
the EXIST operator)

--show the details of orders not
made by customers in germany

SELECT

*

FROM Sales.Orders as o

WHERE NOT EXISTS

(SELECT

*

FROM Sales.Customers as c

where Country = 'Germany'

and o.CustomerID =

c.CustomerID);

--above the results are row by
row evaluated (means the
condition is checked row by
row)

--SUBQUERY SUMMARY

/*

Query inside another QUERY

breakes a complex query into
smaller, manageable pieces

USECASES

1. Create temp result sets

2. Prepare data before joining
TABLE

3. Dynamic and complex
filtering

4. Check the existence of rows
from another table(EXISTS)

5. Row by Row comparison -
corelated subquery

*/

--CTE (COMMON TABLE EXPRESSIONS)

--what we will learn

/*

CTE TYPES

WHY IS IT IMPORTANT

SYNTAXES

EXAMPLES

HOW SQL EXECUTES
CTE's step by step

*/

--CTE

--Temprory named result set
(virtual table), that can be used
mutliple times within your
query to simplify and organize
complex query

/*

Inside a query

there is cte query and a main
query

first between the main and cte query the cte query is executed first

The cte will be having an intermediate virtual table

then the main query will be executed using that virtual table

the main query has two sources of data

1. either get the data directly from the database tables
2. or get the data from the cte virtual table

when everything is done the final result of the main query gonna be presented for the user as a final result

THE INTERMEDIATE VIRTUAL TABLE THAT IS CREATED BY THE CTE HAS 2 FEATURES:

1. It is temporary (once the query ends the sql gonna go and destroy this virtual table) and we will not be able to query it anymore
2. If use join and want to use this virtual table create by cte it will not be working (because its only available locally within the main query)

We can say its somewhere similar to a subquery but there are differences:

1. but its more organized and readable
2. the cte is written before the main query (top to bottom) but subquery (bottom to top)
3. the sub query is introducing an intermediate result that is used later from the main query and the same thing for the cte

MAIN DIFFERENCES BETWEEN CTE AND SUBQUERY

1. In the subquery the result can only be used only once (if we want to use it at multiple places we have to write the subquery multiple times)
2. CTE (we can think this as a virtual table) can be used multiple times within the main query

WHEN TO USE CTE

lets say in a complex sql task we have to do the following:

4. AGGREGATIONS (AVG)
3. JOIN again the same tables in order to prepare the data and perform different aggregations (avg)
2. AGGREGATIONS based on different data (sum)
1. JOIN tables together in order to prepare all the data that we need for the next step

Now normally we were using the subquery

4. AGGREGATIONS (AVG) - Main query
3. JOIN again the same tables - SUBQUERY
2. AGGREGATIONS based on different data (sum) - SUBQUERY
1. JOIN - SUBQUERY

now if we continue to do this we are in a problem

we are repeating the same step more than once (like the JOIN step)

this is the weak point of the subquery it might introduce redundancy and repetition

Now with CTE we can solve this problem

We will just use the following steps with CTE:

1. Join - CTE Query
2. Aggregations (sum) - Main query
3. Aggregations (AVG) - Main query

so the step 1 will be used more than once as being a CTE which will reduce redundancy and repetition and the size of the query

USING THE CTE

we have to select different different stuff

1. like we have to find the top customers

so we will put this in one

CTE_TOP_CUSTOMERS

(select --- from customers where ---)

2. have to get the data of top products

CTE_TOP_PRODUCTS

(select --- from products where ---)

3. have to get the daily revenue

CTE_DAILY_REVENUE

(select --- from orders where -- -)

Once we have all those parts we can put everything together in the main query

select --

So now our code is divided into different sections with increased readability,

organization, Reusability and modularity(breaking queries into parts or chunks)

HOW DATABASE EXECUTES CTE'S STEP BY STEP

1. let suppose making a cte query (with details as(select --- from customers where ---))

2. now we are writing the main query (select -- from orders join details join details join details)

3. the database engine will read the query and will keep the cte at its priority

4. first it will execute the cte query and create an intermediate virtual table (details) and will store the result named DETAILS in the cache memory (very fast memory)

5. now it will execute the main query using the orders table that is in the disk storage and the intermediate virtual table CTE (details) at high speeds because it is stored in the cache memory which is way faster than retrieving data from disk storage like table orders which is stored in disk (hdd or ssd which is slower than cache memory)

CTE TYPES:

1. RECURSIVE CTE

2. NON RECURSIVE CTE - two types

a. SIMPLE CTE (standalone cte) - Define and used independently (runs independently and its self contained and doesn't rely on other CTE's or queries)

means the CTE can query data from the database tables directly and the main query can use the CTE virtual table directly INDEPENDENT FROM ANYTHING ELSE

b. NESTED CTE (cte inside another cte) - uses the result of another CTE, so it can't run independently.

SYNTAX STANDALONE CTE:

```
WITH CTE_NAME As (
select column1, column2, ...
from table_name
where condition
)
select column1, column2, ...
from CTE_NAME
where condition;
```

outside this definition

we can use this cte like above

*/

--STANDALONE CTE

--Find the total sales per customer using CTE

```
WITH CTE_TOTAL_SALES
AS (
```

```
    SELECT
```

```
        CustomerID,
```

```
        SUM(Sales) as Total_Sales
```

```
from Sales.Orders
```

```
GROUP BY CustomerID
```

```
)
```

```
SELECT
```

```
    c.CustomerID,
```

```
    c.FirstName,
```

```
    c.LastName,
```

```
    c.Country,
```

```
    cts.Total_Sales
```

```
from Sales.Customers as c
```

```
LEFT JOIN
```

```
CTE_TOTAL_SALES As cts
```

```
on c.CustomerID =
```

```
cts.CustomerID
```

```
ORDER BY c.CustomerID;
```

--normal way without CTE

```
SELECT
```

```
    CustomerID,
```

```
    SUM(Sales) as Total_Sales
```

```
from Sales.Orders
```

```
GROUP BY CustomerID;
```

--MULTIPLE STANDALONE CTE

--syntax

```
/*
```

```
WITH CTE_NAME As (
```

```
    select column1, column2, ...
```

```
    from table_name
```

```
    where condition
```

```
)
```

```
,CTE_NAME2 As(
```

```
    select column1, column2, ...
```

```
    from table_name
```

```
    where condition
```

```
)
```

```
,CTE_NAME3 As(
```

```
    select column1, column2, ...
```

```
    from table_name
```

```
    where condition
```

```
)
```

```
,CTE_NAME4 As(
```

```
    select column1, column2, ...
```

```
    from table_name
```

```
    where condition
```

```
)
```

```
select column1, column2, ...
```

```
from CTE_NAME
```

```
where condition;
```

```
*/
```

--step1 = find the total sales per customer

--step2 = find the last order date per customer

```
WITH CTE_TOTAL_SALES
as (
```

```
    SELECT
```

```
        CustomerID,
```

```
        SUM(Sales) as TotalSales
```

```
from Sales.Orders
```

```
GROUP BY CustomerID
```

```
)
```

```
, CTE_LAST_ORDER AS (
```

```
    SELECT
```

```
        CustomerID,
```

```
        MAX(OrderDate) as
```

```
LastOrder
```

```
from Sales.Orders
```

```
GROUP BY CustomerID
```

```
)
```

```
SELECT
```

```
    c.CustomerID,
```

```
    CONCAT(c.FirstName, '
',c.LastName) as FullName,
```

```
    cts.TotalSales,
```

```
    clo.LastOrder
```

```
from Sales.Customers as c
```

```
LEFT JOIN
```

```
CTE_TOTAL_SALES AS cts
```

```
on c.CustomerID =
```

```
cts.CustomerID
```

```
LEFT JOIN
```

```
CTE_LAST_ORDER AS clo
```

```
on c.CustomerID =
```

```
clo.CustomerID
```

```
--NESTED CTE (CTE
INSIDE ANOTHER CTE)

--just like nested loops and
nested conditional statements
we have learnt in python

--syntax
/*

WITH CTE_NAME As ( --
STANDALONE CTE

select column1, column2, ...
from table_name
where condition
)

,CTE_NAME2 As( --NESTED
CTE

select column1, column2, ...
from CTE_NAME
where condition
)

SELECT
FROM CTE_NAME2

where----
```

so above basically the
cte_name2 cte is dependent on
the cte_name

cte

```
*/

--step1 = find the total sales
per customer

--step2 = find the last order
date per customer
```

```
--step3 = Rank customer based
on total sales per customer

--step4 = segment customers
based on their total sales

WITH CTE_TOTAL_SALES
as (

    SELECT

        CustomerID,

        SUM(Sales) as TotalSales

    from Sales.Orders

    GROUP BY CustomerID

)

, CTE_LAST_ORDER AS (

    SELECT

        CustomerID,

        MAX(OrderDate) as
LastOrder

    from Sales.Orders

    GROUP BY CustomerID

)

, CTE_RANK_CUSTOMERS
AS (

    SELECT

        CustomerID,

        TotalSales,

        DENSE_RANK() OVER
(ORDER BY TotalSales
DESC) AS CustomerRank

    from CTE_TOTAL_SALES

)

, CTE_CUST_SEGMENT AS
(
```

```
SELECT

    CustomerID,

    CASE

        WHEN TotalSales >
100 THEN 'HIGH'

        WHEN TotalSales
BETWEEN 50 AND 100
THEN 'MED'

        ELSE 'LOW'

    END AS SalesFrequency

    from CTE_TOTAL_SALES

)

SELECT

    c.CustomerID,

    CONCAT(c.FirstName, ' ',
c.LastName) as FullName,

    clo.LastOrder,

    cts.TotalSales,

    crc.CustomerRank,

    ccs.SalesFrequency

    from Sales.Customers as c

    LEFT JOIN
CTE_TOTAL_SALES as cts

    on c.CustomerID =
cts.CustomerID

    LEFT JOIN
CTE_LAST_ORDER as clo

    on c.CustomerID =
clo.CustomerID

    LEFT JOIN
CTE_RANK_CUSTOMERS
as crc

    on c.CustomerID =
crc.CustomerID
```

```

LEFT JOIN
CTE_CUST_SEGMENT as
ccs

on c.CustomerID =
ccs.CustomerID

WHERE cts.TotalSales is not
NULL

ORDER BY
crc.CustomerRank

```

--CTE BEST PRACTICES

```
/*
```

1. Rethink and refactor your cte's before starting a new one (means if one cte can do the work of both the two cte's then there is no sense of making a new one)

2. Dont use more than 5 cte's in a query otherwise our code will be hard to understand and maintain

```
*/
```

--RECURSIVE CTE

--a self refrencing query that repeatedly processes data until a specific condition is met.

```
/*
```

1. The query of the CTE will be executed for the first time and we will get the initial data from the CTE

2. This intermediate result is not ready yet for the main query

3. This intermediate result will go to back to the CTE gonna check whether the current

results is meeting specific conditions or not

4. If not the CTE query gonna be executed second time (its result will be added to the intermediate results) and this loop continues

5. Now intermediate results will have more data

6. Again before we can use it from the main query it is gonna be checked if the result fulfils the condition or not

7. if the condition fulfils the loop gonna end and the result is ready to use from the main query

SYNTAX

```
WITH CTE_NAME As (
```

```
    SELECT... --if we just use
    this one query it will be
    executed only once
```

```
    FROM...
```

```
    WHERE...
```

```
    UNION ALL --in sql we
    cant have 2 sel statements like
    this (so we have to connect it
    using the union or union ALL
    )
```

```
    SELECT...
```

```
    FROM CTE_NAME --here
    we refrenced the cte name to
    itself
```

```
    WHERE [BREAK
    CONDITION] --in order to
    break the loop
```

```
)
```

```
SELECT...
```

```
FROM CTE_NAME
```

```
WHERE...
```

```
OPTION (MAXRECURSION
30) --optional (limit set to
iterate)
```

1. the first query in the with query is called ANCHOR QUERY (will only be executed once)

2. the second query in the with query is called RECURSIVE QUERY (will keep looping until certain condition is met)

3. the second query will keep repeating and add data to the intermediate results until the condition is met (or when there is no more data available to be processed)

```
*/
```

```
--generate a sequence of
numbers from 1 to 20
```

```
WITH Series As (
```

```
--anchor query
```

```
SELECT 1 AS MyNum
```

```
UNION ALL
```

```
--recursive query
```

```
SELECT
```

```

MyNum + 1
FROM Series
WHERE MyNum < 20
)
SELECT *
FROM Series
OPTION (MAXRECURSION
30) --this will make the query
not iterate more than 30

--TASK : Show the employee
hierarchy by displaying each
employee's level within the
organization

WITH CTE_EMP_Hierarchy
as (
    --ANCHOR QUERY

    SELECT
        EmployeeID,
        FirstName,
        ManagerID,
        1 as Level
    from Sales.Employees
    WHERE ManagerID IS
    NULL

    UNION ALL

    --RECURSIVE QUERY
    SELECT
        e.EmployeeID,
        e.FirstName,
        e.ManagerID,
        Level + 1

```

```

FROM Sales.Employees as
e
    INNER JOIN
    CTE_EMP_Hierarchy as ceh
        on e.ManagerID =
        ceh.EmployeeID
)
SELECT
*
FROM
    CTE_EMP_HIERARCHY
/*

1. Anchor query - Ran only
once (which will select the top
level employee not having any
manager)

2. Recursive query - works by
searching if a manager id is
equal to which employee id(
    a. first it will check the 1st
    row which is frank having no
    manager id so the recursive
    query will not find any match
    and will skip it.
    b. then it will move to the
    next row (which is kevin)
    having Frank as manager id so
    it will find a match and add
    kevin to the intermediate
    results with level 2
    c. then it will move to the
    next row (which is mary)
    having frank as manager id so
    it will find a match and add
    mary to the intermediate
    results with level 2
    d. now next iteration will be
    started (as now there are no
    more employees having the

```

frank as their manager id) so
the next level will be 3

e. from the next line
onwards it will keep searching
for employees having either
kevin or mary as their manager
id and will keep adding them
to the intermediate results with
level 3

f. (as the sql stops searching
for the employees having the
ManagerID as (2(kevin),
3(mary)) now the 2rd iteration
will end and 3rd iteration will
start which will search if there
are any employees having the
managerid as (4 Michael), (5
Carol))this will continue until
there are no more employees
left to process (ie no more
matches found)

```

)
*/

/*

```

SUMMARY

CTE is a temporary named
result set that can be used
multiple times within a
QUERY

ADVANTAGES OF CTE:

1. Readability - makes
complex queries easier to read
and understand by breaking
them into smaller, manageable
parts.

2. Modularity - allows you to
define a CTE once and
reference it multiple times

within the same query,
promoting code reuse.

3. Reusability - CTEs can be reused within the same query, reducing redundancy and improving maintainability.

4. Organization - helps organize complex queries by separating different logical parts into distinct sections.

5. Recursion - supports recursive queries, enabling you to work with hierarchical or self-referential data structures.

1. Result of CTE is like Tables which cant be used from multiple queries outside the main query where it is defined

2. TIP : Dont create more than 5 cte in a single query as it may impact performance negatively and readability

*/